INFOSYS SP / DSE CODING ASSESSMENT · 6 DECEMBER 2026

Data Structures and Algorithms in Java

Daily worksheets, exercises and mock exams for the 29-day plan

Volume II · Practice Workbook

Twenty-nine chapters, one per day. Each has the day's plan, a spaced-review list, exercises that you complete by hand, a worksheet for every problem, and an end-of-day error log. Hints and answers are at the back.

Read with: **Volume I · Theory** (same chapter numbers)

Name _____

Prepared for Purvam Prajapati · Edition of 7 November 2026

Contents

1 Java for Problem Solving: Collections, Input and Output 7

2 Prefix Sums, Difference Arrays and Hashing 15

3 Two Pointers and Sliding Windows 22

4 Sorting and Greedy Choice: Intervals 30

5 Greedy with heaps 37

6 Binary Search and Searching on the Answer 41

7 Monotonic Stacks 50

8 Bits and XOR 58

9 Mock 1 + review 62

10 Number Theory for Programmers 65

11 DP I: states, transitions, knapsack 74

12 DP II: two-string DP 78

13 Dynamic Programming III: Increasing Subsequences and Extra Dimensions 82

14 DP IV: divisor, interval and game DP 91

15 Graphs I: BFS, DFS, topological order 95

16 Mock 2 + review 99

17 Graphs II: Dijkstra, DSU, MST 102

18 Trees I: recursion, parent arrays, LCA 106

19 Trees II: tree DP, rerooting, small-to-large 110

20 Fenwick trees and segment trees 114

21 Lazy segment trees and the K-parts DP 118

22 Backtracking, bitmask DP, digit DP 122

23 Mock 3 (4 problems) + review 126

24 Infosys repeat problems I 129

25 Mock 4: the official Infosys sample paper 134

26 Repair day + the hard XOR repeat 138

27 Mock 5: dress rehearsal 141

28 Consolidate 144

29 Rest and logistics 147

A Hints 149

B Answer key 155

C Error log 164

D Mock scores and SP checkpoints 167

E Notes and scratch paper 168

How to use this workbook

Stub.

Progress tracker

Tick a box for each problem solved (alone or after a hint). Shaded boxes are bonus problems.

Day	Date	Topic	Core	Bonus	Hours
1	Sat 7 Nov	Set up, Java toolkit, fast input	☐☐☐☐☐	☐☐☐	
2	Sun 8 Nov	Prefix sums, difference arrays, hashing	☐☐☐☐	☐☐☐	
3	Mon 9 Nov	Two pointers and sliding window	☐☐☐☐	☐☐☐	
4	Tue 10 Nov	Sorting and greedy: intervals	☐☐☐☐	☐☐☐	
5	Wed 11 Nov	Greedy with heaps	☐☐☐☐	☐☐☐	
6	Thu 12 Nov	Binary search, including on the answer	☐☐☐☐	☐☐☐	
7	Fri 13 Nov	Monotonic stacks	☐☐☐☐	☐☐☐	
8	Sat 14 Nov	Bits and XOR	☐☐☐☐	☐☐☐	
9	Sun 15 Nov	Mock 1 + review	☐☐☐		
10	Mon 16 Nov	Number theory toolkit	☐☐☐☐	☐☐☐	
11	Tue 17 Nov	DP I: states, transitions, knapsack	☐☐☐☐	☐☐☐	
12	Wed 18 Nov	DP II: two-string DP	☐☐☐☐	☐☐☐	
13	Thu 19 Nov	DP III: LIS and DP with an extra dimension	☐☐☐	☐☐☐	
14	Fri 20 Nov	DP IV: divisor, interval and game DP	☐☐☐☐	☐☐☐	
15	Sat 21 Nov	Graphs I: BFS, DFS, topological order	☐☐☐☐	☐☐☐	
16	Sun 22 Nov	Mock 2 + review	☐☐☐		
17	Mon 23 Nov	Graphs II: Dijkstra, DSU, MST	☐☐☐☐	☐☐☐	
18	Tue 24 Nov	Trees I: recursion, parent arrays, LCA	☐☐☐☐	☐☐☐☐	
19	Wed 25 Nov	Trees II: tree DP, rerooting, small-to-large	☐☐☐☐	☐☐☐	
20	Thu 26 Nov	Fenwick trees and segment trees	☐☐☐	☐☐☐☐	
21	Fri 27 Nov	Lazy segment trees and the K-parts DP	☐☐☐	☐☐☐	
22	Sat 28 Nov	Backtracking, bitmask DP, digit DP	☐☐☐☐	☐☐☐	
23	Sun 29 Nov	Mock 3 (4 problems) + review	☐☐☐☐		
24	Mon 30 Nov	Infosys repeat problems I	☐☐☐☐☐	☐☐☐☐☐	
25	Tue 1 Dec	Mock 4: the official Infosys sample paper	☐☐☐	☐	
26	Wed 2 Dec	Repair day + the hard XOR repeat	☐☐	☐	
27	Thu 3 Dec	Mock 5: dress rehearsal	☐☐☐☐		
28	Fri 4 Dec	Consolidate	☐☐☐		
29	Sat 5 Dec	Rest and logistics	☐		

Pattern recall tracker

Each pattern comes back for review 1, 3, 7 and 14 days after it is taught; the day numbers are filled in. Write A (again), S (shaky) or G (solid, good) in each cell after the review. Any pattern still at A on Day 26 goes into the repair session.

Pattern	Taught	+1	+3	+7	+14	Notes
Java collections toolkit	1	2	4	8	15	
Fast input and output (FastScanner)	1	2	4	8	15	
Linked list moves	1	2	4	8	15	
Prefix sums + HashMap	2	3	5	9	16	
Difference array (many range adds)	2	3	5	9	16	
Two pointers on a sorted array	3	4	6	10	17	
Sliding window (variable size)	3	4	6	10	17	
Window maximum (monotonic deque)	3	4	6	10	17	
Intervals: sort, then sweep	4	5	7	11	18	
Greedy with a heap	5	6	8	12	19	
Binary search: lower and upper bound	6	7	9	13	20	
Binary search on the answer	6	7	9	13	20	
Monotonic stack	7	8	10	14	21	
Bit tricks and XOR	8	9	11	15	22	
Binary trie for XOR	8	9	11	15	22	
XOR basis (maximum subset XOR)	8	9	11	15	22	
Brute force first, then stress test	9	10	12	16	23	
gcd, fast power, inverse, nCr	10	11	13	17	24	
Sieve, prime factors, square-free kernel	10	11	13	17	24	
DP recipe: state, transition, base, answer	11	12	14	18	25	
Knapsack family	11	12	14	18	25	
Two-string DP (LCS, edit distance)	12	13	15	19	26	
Longest increasing subsequence	13	14	16	20	27	
Interval DP and game DP	14	15	17	21	28	
BFS: fewest edges, grids, many sources	15	16	18	22	29	
DFS without recursion, components	15	16	18	22	29	
Topological order (Kahn) and DAG DP	15	16	18	22	29	
0-1 BFS	15	16	18	22	29	
Dijkstra	17	18	20	24		
Union-find (DSU) and Kruskal	17	18	20	24		
Binary tree recursion (LeetCode TreeNode)	18	19	21	25		
General trees from a parent array (no recursion)	18	19	21	25		
LCA with binary lifting	18	19	21	25		
Tree DP (dp[v][state])	19	20	22	26		
Rerooting (an answer for every root)	19	20	22	26		

Pattern	Taught	+1	+3	+7	+14	Notes
Small-to-large merging	19	20	22	26		
Euler tour (subtree = range)	20	21	23	27		
Fenwick tree (BIT)	20	21	23	27		
Segment tree (iterative)	20	21	23	27		
Lazy segment tree (range add, range max)	21	22	24	28		
Split into K parts (dp[k][i])	21	22	24	28		
Backtracking and subset enumeration	22	23	25	29		
Bitmask DP	22	23	25	29		
Digit DP	22	23	25	29		

DAY 1 OF 29 · SATURDAY 7 NOVEMBER 2026 · LEARNING DAY · SETUP

1

Java for Problem Solving: Collections, Input and Output

Accounts, the Java collections you will use every day, and how FastScanner replaces Scanner.

Read first · Theory Chapter 1
Patterns · Java collections toolkit, Fast input and output (FastScanner), Linked list moves

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 1; type each new listing once and run it	40 min
☐	1.P1 LC 1 · Two Sum	20 min
☐	1.P2 LC 242 · Valid Anagram	20 min
☐	1.P3 LC 20 · Valid Parentheses	20 min
☐	1.P4 LC 347 · Top K Frequent Elements	40 min
☐	1.P5 LC 729 · My Calendar I	40 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 1.P6, 1.P7, 1.P8*	*60 min*
	Core total (estimate)	**3.9 h**

NOTES FOR THE DAY

- Submit the Infosys date/location form (deadline **8 Nov**) and choose 6 Dec.
- Create free accounts on LeetCode and Codeforces. In LeetCode's editor, set the language to Java.
- Read Theory Chapter 1 (fast input): why Scanner is slow and how FastScanner works. Type FastScanner once from memory, paste it into OneCompiler, put `3 10 20 30` in the STDIN box and check that it prints 60.
- Read the handbook cards below. From tomorrow, the handbook's "Today's list" tells you what to review.
- Solve the problems in LeetCode's editor: click *Run* first, then *Submit*.

Why it matters for Infosys. The exam gives you a plain editor and standard input. Being quick with Java collections and input reading saves minutes on every later problem.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Exercises

Work these after reading Theory Chapter 1 and before the problem worksheets. The first three practise the templates you will type all month (the input reader, the stack check and the ordered-map look-up); the rest test the Java details that silently cost marks and ask you to design and debug small programs of your own.

EXERCISE 1.1 Fill the gaps: FastScanner

Complete the reader from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 1.6*, type it into OneCompiler, put `3 10 20 30` in the STDIN box and check that it prints 60.

```
String next() throws IOException {
    while (_______________________________) {        // (1) current line used up
        String line = br.readLine();
        if (line == null) _______________;           // (2) end of input
        st = _______________________________;        // (3) split the new line
    }
    return st.nextToken();
}

public static void main(String[] args) throws IOException {
    FastScanner fs = new FastScanner(System.in);
    PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
    int n = fs.nextInt();
    long sum = 0;
    for (int i = 0; i < n; i++) sum += _______________;   // (4)
    out.println(sum);
    _______________;                                     // (5) or nothing is printed
}
```

Why does the same program work whether the four numbers are on one line or on four lines?

EXERCISE 1.2 Trace the stack check

Trace *Theory Listing 1.2* on `([]{()})]`. For each character write the action (push which closer, or pop which closer and whether it matches) and the stack afterwards, from bottom to top. Stop at the row where the method returns.

char	action	stack (bottom → top)
(		
[		
]		
{		
(		
)		
}		
)		
]		

Which branch of the `else if` returns `false` here, and what would the method return for `([]{()})` (the same string without the last character)?

EXERCISE 1.3 A faded trace: My Calendar

Run *Theory Listing 1.4* on the requests below, in order. The first three rows are done. For each request write the floor and ceiling neighbours of its start as start (end), and whether it is accepted.

request	floorKey (end)	ceilingKey (end)	accepted?
[3, 8)	null	null	yes
[8, 12)	3 (8)	null	yes
[1, 4)	null	3 (8)	no: [3, 8) starts before 4
[12, 15)			
[10, 13)			
[0, 3)			
[15, 16)			
[14, 16)			

Write the final contents of the map.

EXERCISE 1.4 Concept check: what does Java do?

For each line, write the value or the outcome, and one line of reason. Do it on paper first; then run the lines to check.

1. `int x = 46341 * 46341;`
2. `-10 % 4` and `Math.floorMod(-10, 4)`
3. `Integer a = 127, b = 127; a == b` and `Integer c = 200, d = 200; c == d`
4. `String t = "ab"; t += "c";` then `t == "abc"` and `t.equals("abc")`
5. `List<Integer> l = new ArrayList<>(Arrays.asList(5, 6, 7)); l.remove(1);` What is `l`?
6. `new ArrayDeque<Integer>().push(null);`
7. `'a' + 'b'` and `(char) ('a' + 25)`
8. Printing a `PriorityQueue` shows its elements in sorted order. True or false?

EXERCISE 1.5 Complexity and limits

For each situation, say whether the plan fits in about one second of Java, using *Theory Table 1.2* and the figure of 10^8–$3 \cdot 10^8$ basic steps per second. Show the estimate.

1. $n = 2 \cdot 10^5$; check every pair $i < j$.
2. $n = 20$; try every subset, doing $O(n)$ work per subset.
3. $n = 10^5$; sort, then one binary search per element.
4. $n = 3\,000$; three nested loops over all triples (i, j, k).
5. A $5\,000 \times 5\,000$ table of `long` values: how much memory?

EXERCISE 1.6 Design before you code: a parking garage

A garage has spots numbered $1..m$ ($m \le 10^5$). There are $q \le 2 \cdot 10^5$ events. "Arrive p": a car wants the smallest free spot with number $\ge p$; it parks there, or is turned away (print -1) if none is free. "Leave s": the car in spot s leaves. Choose the data structure, the call that answers "arrive", and the total complexity. Then process these events for $m = 6$ and write the output of each arrival.

`arrive 2, arrive 2, arrive 5, arrive 5, leave 3, arrive 2, arrive 6, arrive 1, leave 6, arrive 4`

EXERCISE 1.7 Find the bug

This method should say whether the values x and y occur equally often in a. It passes small tests. Find an input on which it returns the wrong answer, explain why, and fix it with one change.

```
static boolean sameFrequency(int[] a, int x, int y) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int v : a) cnt.merge(v, 1, Integer::sum);
    return cnt.get(x) == cnt.get(y);
}
```

EXERCISE 1.8 Write it from memory: list moves

Without looking at *Theory Listing 1.8*, write `reverse` and `hasCycle`. Then answer: by *Theory Theorem 1.11*, after how many steps do slow and fast meet on a list with a tail of $\mu = 4$ nodes and a cycle of $\lambda = 6$? With $\mu = 10$ and $\lambda = 3$?

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 5 core problems, 3 bonus.

1.P1 **LC 1 · Two Sum** EASY CORE — **20 min**

`leetcode.com/problems/two-sum · LeetCode`

Pattern	**Target** O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 1.P1*

1.P2 **LC 242 · Valid Anagram** EASY CORE — **20 min**

`leetcode.com/problems/valid-anagram · LeetCode`

Pattern	**Target** O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 1.P2*

1.P3 LC 20 · Valid Parentheses EASY CORE

20 min

leetcode.com/problems/valid-parentheses · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 1.P3

1.P4 LC 347 · Top K Frequent Elements MEDIUM CORE

40 min

leetcode.com/problems/top-k-frequent-elements · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 1.P4

1.P5 LC 729 · My Calendar I MEDIUM CORE

40 min

leetcode.com/problems/my-calendar-i · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 1.P5

1.P6 LC 206 · Reverse Linked List EASY BONUS

20 min

leetcode.com/problems/reverse-linked-list · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 1.P6

1.P7 LC 141 · Linked List Cycle EASY BONUS — **20 min**

`leetcode.com/problems/linked-list-cycle · LeetCode`

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 1.P7*

1.P8 LC 49 · Group Anagrams MEDIUM BONUS — **40 min**

`leetcode.com/problems/group-anagrams · LeetCode`

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 1.P8*

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 2, Prefix sums, difference arrays, hashing.

DAY 2 OF 29 · SUNDAY 8 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

2

Prefix Sums, Difference Arrays and Hashing

Count subarrays by remembering earlier prefix values in a HashMap.

Read first · Theory Chapter 2
Patterns · Prefix sums + HashMap, Difference array (many range adds)

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 2; type each new listing once and run it	40 min
☐	2.P1 LC 560 · Subarray Sum Equals K	40 min
☐	2.P2 LC 974 · Subarray Sums Divisible by K	40 min
☐	2.P3 LC 1524 · Number of Sub-arrays With Odd Sum	40 min
☐	2.P4 LC 525 · Contiguous Array	40 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 2.P5, 2.P6, 2.P7*	*60 min*
	Core total (estimate)	**3.8 h**

NOTES FOR THE DAY

- `P[i] = a[0] + … + a[i-1]`, so `sum(l..r) = P[r+1] - P[l]`. Use `long`.
- Counting subarrays with sum k: at each index, count earlier prefixes equal to `P - k`. Seed the map with `{0: 1}`.
- For "divisible by k", store `Math.floorMod(P, k)`; Java's `%` can be negative.
- Many range adds, read once: `d[l] += v; d[r+1] -= v`, then take a running sum.

Why it matters for Infosys. Q1 is often an array problem with a counting twist. Prefix sums + HashMap is one of the most reported Infosys patterns.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Java collections toolkit	Ch. 1	1 day ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Fast input and output (FastScanner)	Ch. 1	1 day ago	☐ Again ☐ Shaky ☐ Solid
Linked list moves	Ch. 1	1 day ago	☐ Again ☐ Shaky ☐ Solid

Exercises

Work these after reading Theory Chapter 2 and before the problem worksheets. They move from a complete worked example (in the theory book) to a faded one with gaps, and then to questions you answer on your own.

EXERCISE 2.1 Fill the gaps

Complete the template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 2.2*.

```
static long countSubarraysWithSum(int[] a, long k) {
    Map<Long, Integer> seen = new HashMap<>();
    ______________________________;          // (1) the empty prefix
    long prefix = 0, count = 0;
    for (int x : a) {
        prefix += x;
        count += ______________________________;   // (2) earlier prefixes that match
        ______________________________;            // (3) record this prefix
    }
    return count;
}
```

Why must line (2) come before line (3)?

EXERCISE 2.2 Trace the prefix-count scan

Trace *Theory Listing 2.2* on a = [3, 4, −7, 3, 1, 3, 1, −4, −2, −2] with k = 7. The map starts as {0: 1}. For each row, write the prefix after reading $a[i]$, the value looked up, how many earlier prefixes matched, and the running total.

i	a[i]	prefix	prefix − k	found	total
0	3				
1	4				
2	−7				
3	3				
4	1				
5	3				

i	a[i]	prefix	prefix − k	found	total
6	1				
7	−4				
8	−2				
9	−2				

Row 5 finds more than one match. Which subarrays ending at index 5 have sum 7?

EXERCISE 2.3 A faded trace: remainders

Count the subarrays of $a = [4, 5, 0, -2, -3, 1]$ whose sum is divisible by $k = 5$, using *Theory Listing 2.3*. The first three rows are done. Fill in the rest. Use `Math.floorMod`, so every remainder is in 0..4.

i	a[i]	remainder of prefix	cnt[remainder] before	total	cnt after this row
0	4	4	0	0	[1, 0, 0, 0, 1]
1	5	4	1	1	[1, 0, 0, 0, 2]
2	0	4	2	3	[1, 0, 0, 0, 3]
3	−2				
4	−3				
5	1				

At which row would `(prefix + x) % 5` (plain `%`) give a negative value, and what would happen?

EXERCISE 2.4 Difference array by hand

An array of six zeros receives four updates: +2 on [0..3], +5 on [2..2], −1 on [1..5] and +3 on [4..5]. Fill in the difference array d (seven slots) after all updates, then the final array a by running sums. Check one entry of a by counting the updates that cover it.

index	0	1	2	3	4	5	6
d							
a							—

EXERCISE 2.5 Two-dimensional prefix sums

For the grid below, fill in the prefix table S (with its row and column of zeros), then compute the sum of rows 1..2 and columns 1..2 with the formula of *Theory Theorem 2.4*. Check it by adding the four cells directly.

g	c0	c1	c2
r0	3	0	1
r1	5	6	3
r2	1	2	0

S	0	1	2	3
0	0	0	0	0
1	0			
2	0			
3	0			

Sum of rows 1..2, columns 1..2 = S[3][3] − S[1][3] − S[3][1] + S[1][1] =

EXERCISE 2.6 Concept check

Answer true or false, and give a one-line reason.

1. If every element is positive, a sliding window can count subarrays with sum k in $O(n)$.
2. For "longest subarray with sum k", the map should store the latest index of each prefix.
3. `HashMap<Long, Integer>` is a better choice than `int[]` when the keys are remainders $0..k-1$ with $k \le 10^6$.
4. A difference array can answer "what is $a[4]$ now?" between two updates in $O(1)$.
5. The number of odd-sum subarrays equals (number of even prefixes) × (number of odd prefixes), with $P[0]$ counted as even.

EXERCISE 2.7 Design before you code

A binary string s of length up to 10^5 is given. Count the substrings that contain as many 0s as 1s. Using *Theory Table 2.1*, say what you store for each prefix, what you look up, and whether you store a count or an index. Then compute the answer for $s =$ `0011010` by hand.

EXERCISE 2.8 **Find the bug**

This method is meant to solve LeetCode 974, but it fails on some inputs. Find the input type that breaks it, give a failing example, and fix it with one change.

```
static int subarraysDivByKBuggy(int[] a, int k) {
    int[] cnt = new int[k];
    cnt[0] = 1;
    int prefix = 0, total = 0;
    for (int x : a) {
        prefix = (prefix + x) % k;
        if (prefix < 0) break;          // "can't happen"
        total += cnt[prefix];
        cnt[prefix]++;
    }
    return total;
}
```

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

2.P1 **LC 560 · Subarray Sum Equals K** MEDIUM CORE **40 min**

leetcode.com/problems/subarray-sum-equals-k · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 2.P1*

2.P2 LC 974 · Subarray Sums Divisible by K

MEDIUM CORE

40 min

leetcode.com/problems/subarray-sums-divisible-by-k · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 2.P2

2.P3 LC 1524 · Number of Sub-arrays With Odd Sum

MEDIUM CORE

40 min

leetcode.com/problems/number-of-sub-arrays-with-odd-sum · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 2.P3

2.P4 LC 525 · Contiguous Array

MEDIUM CORE

40 min

leetcode.com/problems/contiguous-array · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 2.P4

2.P5 LC 1109 · Corporate Flight Bookings

MEDIUM BONUS

40 min

leetcode.com/problems/corporate-flight-bookings · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 2.P5

2.P6 **LC 304 · Range Sum Query 2D - Immutable** MEDIUM BONUS **40 min**

leetcode.com/problems/range-sum-query-2d-immutable · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 2.P6*

2.P7 **CF 466C · Number of Ways** CF 1700 BONUS **45 min**

codeforces.com/problemset/problem/466/C · Codeforces

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 2.P7*

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 3, Two pointers and sliding window.

DAY 3 OF 29 · MONDAY 9 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

3

Two Pointers and Sliding Windows

Grow the window on the right; shrink it on the left while it is invalid.

Read first · Theory Chapter 3
Patterns · Two pointers on a sorted array, Sliding window (variable size), Window maximum (monotonic deque)

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 3; type each new listing once and run it	40 min
☐	3.P1 LC 3 · Longest Substring Without Repeating Characters	40 min
☐	3.P2 LC 1004 · Max Consecutive Ones III	40 min
☐	3.P3 LC 424 · Longest Repeating Character Replacement	40 min
☐	3.P4 LC 15 · 3Sum	40 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 3.P5, 3.P6, 3.P7*	*60 min*
	Core total (estimate)	**3.8 h**

NOTES FOR THE DAY

- A sliding window works when shrinking a valid window keeps it valid.
- Template: `for r: add(a[r]); while (bad) remove(a[l++]); best = max(best, r - l + 1);`
- "Exactly K" = atMost(K) − atMost(K − 1).
- Sorted array with a target sum: two pointers from both ends; skip equal values to avoid duplicates.

Why it matters for Infosys. The Infosys "Andy's vacation" problem (cancel at most K obligations) is LeetCode 1004 on a calendar.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Prefix sums + HashMap	Ch. 2	1 day ago	☐ Again ☐ Shaky ☐ Solid
Difference array (many range adds)	Ch. 2	1 day ago	☐ Again ☐ Shaky ☐ Solid

Exercises

Work these after reading Theory Chapter 3 and before the problem worksheets. The first three are the window and deque templates by hand; then you test rules for monotonicity, count "exactly K" windows, design a two-deque window of your own and find a bug that only shows up on some inputs.

EXERCISE 3.1 Fill the gaps: the window template

Complete the template from memory, then compare with *Theory Listing 3.3*. Each blank is one expression or statement.

```
static int longestAtMostKDistinct(int[] a, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    int best = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        ________________________________;                    // (1) add a[r]
        while (________________________) {                   // (2) the window is invalid
            if (cnt.merge(a[l], -1, Integer::sum) == 0) _____________;   // (3)
            ______;                                          // (4)
        }
        best = ________________________________;             // (5)
    }
    return best;
}
```

Which line would you change to count all valid windows instead of finding the longest, and how?

EXERCISE 3.2 Trace the monotonic deque

Trace *Theory Listing 3.9* on $a = [3, 1, 5, 2, 2, 6, 0, 4]$ with $k = 3$. Write the **indices** in the deque after each step, front first, and the window maximum once $i \geq 2$.

i	a[i]	indices removed at back	index removed at front	deque (indices)	max
0	3				—
1	1				—
2	5				
3	2				
4	2				
5	6				
6	0				
7	4				

At $i = 4$ the new value equals the value at the back. What does the `<=` in the listing do with the older copy, and why is that safe?

EXERCISE 3.3 A faded trace: at most two distinct values

Run *Theory Listing 3.3* on $a = [1, 2, 1, 3, 3, 2, 2, 4]$ with $k = 2$. The first three rows are done; fill in the rest.

r	a[r]	l before → after	counts in the window	length	best
0	1	0 → 0	{1: 1}	1	1
1	2	0 → 0	{1: 1, 2: 1}	2	2
2	1	0 → 0	{1: 2, 2: 1}	3	3
3	3				
4	3				
5	2				
6	2				
7	4				

EXERCISE 3.4 Concept check: is the rule monotone?

For each rule, say whether it is monotone in the sense of *Theory Definition 3.2* (every sub-window of a valid window is valid), and give a one-line reason or a counterexample. Then say which loop you would use: grow-and-shrink, shrink-while-valid, or neither.

1. The window contains at most 2 zeros.
2. The window's sum is at least 10 (all values positive).
3. The window's sum is exactly 10 (all values positive).
4. The window's maximum minus its minimum is at most 3.
5. The window contains at least one copy of every vowel.
6. The window's sum is at most 10 (values may be negative).

EXERCISE 3.5 Exactly K by subtraction

Count the subarrays of $a = [2, 2, 1, 3, 1]$ that contain exactly two distinct values, as atMost(2) − atMost(1). For each scan, write l after shrinking and the number of windows added at each r. Then list the subarrays to check.

r	a[r]	atMost(2): l	added	atMost(1): l	added
0	2				
1	2				
2	1				
3	3				
4	1				
total					

Exactly two distinct values: atMost(2) − atMost(1) =

EXERCISE 3.6 Design before you code: a window with two deques

Given $a[0..n-1]$ with $n \le 10^5$ and an integer $d \ge 0$, find the longest subarray whose maximum and minimum differ by at most d. Show that the rule is monotone, say what state the window needs so that "max − min" is available in O(1) amortised, describe what happens when l advances, and give the complexity. Then find the answer for $a = [4, 6, 5, 9, 7, 8, 3, 8]$ and $d = 2$.

EXERCISE 3.7 Find the bug

This version of the template passed both samples it was tried on. Find an input on which it fails, say how it fails, and fix it.

```
static int longestAtMostKDistinctBuggy(int[] a, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    int best = 0;
```

```
    for (int l = 0, r = 0; r < a.length; r++) {
        cnt.merge(a[r], 1, Integer::sum);
        while (cnt.size() > k) {
            cnt.merge(a[l], -1, Integer::sum);
            l++;
        }
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

EXERCISE 3.8 **Complexity and limits: which tool?**

For each problem, name the technique from *Theory Table 3.1*, the target complexity, and the one fact about the input that decides it.

1. $n \le 10^5$, values in $[-10^4, 10^4]$: count subarrays with sum exactly k.
2. $n \le 10^5$, values in $[1, 10^4]$: the shortest subarray with sum $\ge S$.
3. $n \le 3\,000$: count the triples $i < j < k$ with $a[i] + a[j] + a[k] < T$.
4. $n \le 10^6$, $k \le n$: the minimum of every window of size k.
5. A string of length $\le 10^5$: the longest substring with at most 3 distinct characters.

EXERCISE 3.9 **Write it from memory: the window maximum**

Without looking, write `windowMax` with the monotonic deque. Then state, in two sentences, the invariant that makes the front the maximum and the reason the total work is at most $2n$ deque operations.

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

3.P1 LC 3 · Longest Substring Without Repeating Characters MEDIUM CORE — 40 min

`leetcode.com/problems/longest-substring-without-repeating-characters` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P1*

3.P2 LC 1004 · Max Consecutive Ones III MEDIUM CORE — 40 min

`leetcode.com/problems/max-consecutive-ones-iii` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P2*

3.P3 LC 424 · Longest Repeating Character Replacement MEDIUM CORE — 40 min

leetcode.com/problems/longest-repeating-character-replacement · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P3*

3.P4 LC 15 · 3Sum MEDIUM CORE — 40 min

leetcode.com/problems/3sum · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P4*

3.P5 LC 992 · Subarrays with K Different Integers HARD BONUS — 60 min

leetcode.com/problems/subarrays-with-k-different-integers · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P5*

3.P6 LC 239 · Sliding Window Maximum HARD BONUS — 60 min

leetcode.com/problems/sliding-window-maximum · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P6*

3.P7 **LC 76 · Minimum Window Substring** HARD BONUS **60 min**

`leetcode.com/problems/minimum-window-substring` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P7*

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 4, Sorting and greedy: intervals.

DAY 4 OF 29 · TUESDAY 10 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

4

Sorting and Greedy Choice: Intervals

Sort by the right key, then make one decision per element. Check the greedy by swapping two choices.

Read first · Theory Chapter 4
Patterns · Intervals: sort, then sweep

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 4; type each new listing once and run it	40 min
☐	4.P1 LC 56 · Merge Intervals	40 min
☐	4.P2 LC 435 · Non-overlapping Intervals	40 min
☐	4.P3 LC 2406 · Divide Intervals Into Minimum Number of Groups	40 min
☐	4.P4 LC 179 · Largest Number	40 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 4.P5, 4.P6, 4.P7*	*60 min*
	Core total (estimate)	**3.8 h**

NOTES FOR THE DAY

- Exchange argument: if swapping two neighbouring choices never helps, the sort order is optimal.
- Most non-overlapping intervals: sort by END. Fewest platforms or rooms: the busiest moment, found with a sweep.
- Comparators: `Arrays.sort(a, (x, y) -> Integer.compare(x[1], y[1]))`, never `x[1] - y[1]`.
- Largest number: compare `b + a` with `a + b` as strings.

Why it matters for Infosys. Q2 is often a greedy problem. Minimum platforms and Largest number are both on the most-reported Infosys list.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Two pointers on a sorted array	Ch. 3	1 day ago	☐ Again ☐ Shaky ☐ Solid
Sliding window (variable size)	Ch. 3	1 day ago	☐ Again ☐ Shaky ☐ Solid
Window maximum (monotonic deque)	Ch. 3	1 day ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Java collections toolkit	Ch. 1	3 days ago	☐ Again ☐ Shaky ☐ Solid
Fast input and output (FastScanner)	Ch. 1	3 days ago	☐ Again ☐ Shaky ☐ Solid
Linked list moves	Ch. 1	3 days ago	☐ Again ☐ Shaky ☐ Solid

Exercises

Work these after reading Theory Chapter 4 and before the problem worksheets. They practise the three interval templates, then ask you to write an exchange argument, refute a false greedy, order strings for the Largest Number problem and design a greedy of your own.

EXERCISE 4.1 Fill the gaps: merge and schedule

Complete both templates from memory, then compare with *Theory Listing 4.3* and *Theory Listing 4.4*.

```
static int[][] merge(int[][] iv) {
    Arrays.sort(iv, (x, y) -> _________________________________);    // (1) sort key
    List<int[]> out = new ArrayList<>();
    for (int[] cur : iv) {
        int[] last = out.isEmpty() ? null : out.get(out.size() - 1);
        if (last != null && _______________) last[1] = _______________;   // (2) (3)
        else out.add(new int[]{cur[0], cur[1]});
    }
    return out.toArray(new int[0][]);
}

static int maxNonOverlapping(int[][] iv) {
    Arrays.sort(iv, (x, y) -> _________________________________);    // (4) sort key
    int count = 0;
    long end = Long.MIN_VALUE;
    for (int[] cur : iv) {
        if (_______________) { count++; end = cur[1]; }            // (5) touching allowed
    }
    return count;
}
```

Why does `merge` sort by start while `maxNonOverlapping` sorts by end? What does `merge` return for [1, 2], [2, 3]?

EXERCISE 4.2 Trace the event sweep

Six meetings are closed intervals: [1, 4], [2, 3], [4, 6], [6, 8], [3, 5], [7, 9]. List the twelve events in the order *Theory Listing 4.6* processes them (equal times: starts first), with the running count.

time	event	busy	best	time	event	busy	best

time	event	busy	best	time	event	busy	best

How many rooms are needed? How many if a room freed at time t may be used by a meeting starting at time t?

EXERCISE 4.3 **A faded trace: the heap of end times**

Run *Theory Listing 4.7* on the same six meetings. They are processed in order of start; the first two rows are done.

meeting	new room or reuse (freed at)	heap after (sorted)
[1, 4]	new room	{4}
[2, 3]	new room	{3, 4}
[3, 5]		
[4, 6]		
[6, 8]		
[7, 9]		

EXERCISE 4.4 **Write an exchange argument**

One machine must run n jobs, one at a time; job i takes t_i minutes. The *completion time* of a job is the minute it finishes. Show with an exchange of two **neighbouring** jobs that running the jobs in increasing order of t minimises the sum of completion times. Then check by listing all six orders of t = (3, 1, 2) with their sums.

EXERCISE 4.5 Refute a greedy

"To pay an amount with the fewest coins, always take the largest coin that fits." Find the smallest amount for which this fails with coins {1, 3, 4}, and give the greedy's coins and an optimal set. Then name a coin system for which you believe the rule is correct, and say how you would gain confidence without a proof.

EXERCISE 4.6 Order strings for the Largest Number

Sort "8", "89", "898", "9" and "0" with the comparator of *Theory Section 4.6*. For each neighbouring pair in your answer, show the two concatenations you compared. Then compute $v(x)$ for "89" and "898" and check that it agrees. What would the method return for [0, 0]?

EXERCISE 4.7 Design before you code: the fewest coins to add

You have coins, sorted, and a target $n \le 10^9$. Find the fewest coins you must **add** (any values you like) so that every total $1..n$ can be paid with a subset of all the coins. Use *Theory Lemma 4.7*: describe the state, the step when the next coin fits, the step when it does not, and why the number of added coins is $O(\log n)$. Then run your method on coins [1, 5, 10] with $n = 20$.

EXERCISE 4.8 **Concept check: sorting in Java**

True or false, with a one-line reason.

1. `Arrays.sort(int[])` is stable.
2. `Arrays.sort(Integer[])` takes O(n log n) time in the worst case on every JDK.
3. `(x, y) -> x[0] - y[0]` is a correct comparator whenever all values are between -10^9 and 10^9.
4. Sorting 10^6 `Integer` objects costs about the same as sorting an `int[]` of the same size.
5. A comparison sort can sort any 5 distinct values with at most 6 comparisons.
6. If a greedy matches a brute force on 3,000 random inputs of size 4, it is correct.

EXERCISE 4.9 **Two passes by hand**

Apply *Theory Listing 4.8* to h = [2, 5, 5, 0, 3, 4, 1]: write the array after each pass, then check that neighbours differ by at most 1 and that no entry exceeds its cap.

index	0	1	2	3	4	5	6
h	2	5	5	0	3	4	1
after pass 1							
after pass 2							

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

4.P1 **LC 56 · Merge Intervals** MEDIUM CORE **40 min**

`leetcode.com/problems/merge-intervals` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 4.P1*

4.P2 LC 435 · Non-overlapping Intervals MEDIUM CORE — 40 min

leetcode.com/problems/non-overlapping-intervals · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 4.P2

4.P3 LC 2406 · Divide Intervals Into Minimum Number of Groups MEDIUM CORE — 40 min

leetcode.com/problems/divide-intervals-into-minimum-number-of-groups · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 4.P3

4.P4 LC 179 · Largest Number MEDIUM CORE — 40 min

leetcode.com/problems/largest-number · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 4.P4

4.P5 LC 1798 · Maximum Number of Consecutive Values You Can Make MEDIUM BONUS — 40 min

leetcode.com/problems/maximum-number-of-consecutive-values-you-can-make · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 4.P5

4.P6 LC 452 · Minimum Number of Arrows to Burst Balloons MEDIUM BONUS

40 min

leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 4.P6

4.P7 LC 135 · Candy HARD BONUS

60 min

leetcode.com/problems/candy · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 4.P7

Re-solve from three days ago

Pick one problem from Day 1 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 1.P1 LC 1 · Two Sum ☐ 1.P2 LC 242 · Valid Anagram ☐ 1.P3 LC 20 · Valid Parentheses

☐ 1.P4 LC 347 · Top K Frequent Elements ☐ 1.P5 LC 729 · My Calendar I

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 5, Greedy with heaps.

DAY 5 OF 29 · WEDNESDAY 11 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

5

Greedy with heaps

Unlock options in sorted order, keep the unlocked ones in a heap, undo your worst choice when a limit breaks.

Read first · Theory Chapter 5
Patterns · Greedy with a heap

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 5; type each new listing once and run it	40 min
☐	5.P1 LC 502 · IPO	60 min
☐	5.P2 LC 1642 · Furthest Building You Can Reach	40 min
☐	5.P3 LC 1526 · Minimum Number of Increments on Subarrays to Form a Target Array	60 min
☐	5.P4 LC 1046 · Last Stone Weight	20 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 5.P5, 5.P6, 5.P7*	*60 min*
	Core total (estimate)	**4.1 h**

NOTES FOR THE DAY

- "Best among the options unlocked so far": sort by the unlock key and push options into a `PriorityQueue` as they unlock.
- Regret greedy: take everything, remember it in a heap, and when you break a limit, drop the worst item.
- Max-heap: `new PriorityQueue<>(Collections.reverseOrder())`.
- Fewest "subtract 1 on a subarray" operations to reach zero = Σ max(0, a[i] − a[i−1]).

Why it matters for Infosys. The RPG monsters problem (11 reports) is this pattern. The Infosys "make the array zero" problem builds on LeetCode 1526.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Intervals: sort, then sweep	Ch. 4	1 day ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Prefix sums + HashMap	Ch. 2	3 days ago	☐ Again ☐ Shaky ☐ Solid
Difference array (many range adds)	Ch. 2	3 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

5.P1 **LC 502 · IPO** HARD CORE — **60 min**

`leetcode.com/problems/ipo` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 5.P1*

5.P2 **LC 1642 · Furthest Building You Can Reach** MEDIUM CORE — **40 min**

`leetcode.com/problems/furthest-building-you-can-reach` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 5.P2*

5.P3 **LC 1526 · Minimum Number of Increments on Subarrays to Form a Target Array** HARD CORE — **60 min**

`leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 5.P3*

5.P4 LC 1046 · Last Stone Weight EASY CORE — 20 min

leetcode.com/problems/last-stone-weight · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 5.P4*

5.P5 LC 871 · Minimum Number of Refueling Stops HARD BONUS — 60 min

leetcode.com/problems/minimum-number-of-refueling-stops · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 5.P5*

5.P6 LC 630 · Course Schedule III HARD BONUS — 60 min

leetcode.com/problems/course-schedule-iii · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 5.P6*

5.P7 CF 1526C2 · Potions (Hard Version) CF 1600 BONUS — 45 min

codeforces.com/problemset/problem/1526/C2 · Codeforces

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 5.P7*

Re-solve from three days ago

Pick one problem from Day 2 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 2.P1 LC 560 · Subarray Sum Equals K ☐ 2.P2 LC 974 · Subarray Sums Divisible by K

☐ 2.P3 LC 1524 · Number of Sub-arrays With Odd Sum ☐ 2.P4 LC 525 · Contiguous Array

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 6, Binary search, including on the answer.

DAY 6 OF 29 · THURSDAY 12 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

6

Binary Search and Searching on the Answer

When "can the answer be x?" is monotone, binary search x and check it greedily.

Read first · Theory Chapter 6
Patterns · Binary search: lower and upper bound, Binary search on the answer

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 6; type each new listing once and run it	40 min
☐	6.P1 LC 34 · Find First and Last Position of Element in Sorted Array	40 min
☐	6.P2 LC 875 · Koko Eating Bananas	40 min
☐	6.P3 LC 1011 · Capacity To Ship Packages Within D Days	40 min
☐	6.P4 LC 410 · Split Array Largest Sum	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 6.P5, 6.P6, 6.P7*	*60 min*
	Core total (estimate)	**4.1 h**

NOTES FOR THE DAY

- Write `ok(x)` first, usually greedy and O(n), then binary search the smallest x that passes.
- Use `long` bounds and `lo + (hi - lo) / 2`.
- Write lower and upper bound yourself; `Arrays.binarySearch` returns any matching index.
- "Minimise the maximum" and "maximise the minimum" almost always mean binary search on the answer.

Why it matters for Infosys. Binary search on the answer is one of the most reported pattern families in the Infosys question bank.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Greedy with a heap	Ch. 5	1 day ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Two pointers on a sorted array	Ch. 3	3 days ago	☐ Again ☐ Shaky ☐ Solid
Sliding window (variable size)	Ch. 3	3 days ago	☐ Again ☐ Shaky ☐ Solid
Window maximum (monotonic deque)	Ch. 3	3 days ago	☐ Again ☐ Shaky ☐ Solid

Exercises

Work these after reading Theory Chapter 6 and before the problem worksheets. The first three rebuild the two templates by hand, the middle ones ask you to design and debug, and the last ones are retrieval practice.

EXERCISE 6.1 Fill the gaps

Complete the lower-bound template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 6.1*.

```
// First index with a[i] >= x (a.length if none). a must be sorted.
static int lowerBound(int[] a, int x) {
    int lo = 0, hi = _____________;              // (1)
    while (_____________) {                      // (2)
        int mid = _____________;                 // (3) no overflow
        if (a[mid] >= x) _____________;          // (4)
        else _____________;                      // (5)
    }
    return lo;
}
```

Why does `hi` start at `a.length` and not at `a.length - 1`? Give an input where the second choice returns a wrong answer.

EXERCISE 6.2 Trace two searches

Let a = [2, 4, 4, 6, 6, 6, 9, 11, 15] (n = 9). Trace `upperBound(a, 8)` and `lowerBound(a, 6)` from *Theory Listing 6.1*. Write one row per iteration; leave unused rows empty.

upperBound(a, 8)	lo	hi	mid	a[mid]	next
1					
2					
3					
4					
5					

lowerBound(a, 6)	lo	hi	mid	a[mid]	next
1					
2					
3					
4					
5					

How many times does 6 occur, using only lower and upper bounds? What could `Arrays.binarySearch(a, 6)` return, and what does `Arrays.binarySearch(a, 5)` return?

EXERCISE 6.3 A faded trace: ship capacity

Run `minCapacity` (*Theory Listing 6.5*) on the weights [3, 2, 2, 4, 1, 4] with 3 days. The first rows are done. Complete the trace, then write the greedy days for the answer.

lo	hi	mid	daysNeeded(mid)	decision
4	16	10	2	works: hi = 10
4	10	7	3	works: hi = 7

Answer and the greedy days for it:

EXERCISE 6.4 Concept check

Answer true or false, and give a one-line reason.

1. `Arrays.binarySearch` returns the first index holding the key when the key occurs several times.
2. `lowerBound(a, x)` can return `a.length`.
3. In a "maximise the minimum" search with the branches `lo = mid` and `hi = mid - 1`, the midpoint must be rounded down.
4. If ok(x) is monotone on a range of V values, the search calls ok at most $\lceil \log_2 V \rceil$ times.
5. For a real-valued answer near 10^9, `while (hi - lo > 1e-9)` is a safe stopping rule.
6. The value found by "smallest x with count(x) $\geq k$" might not be an element of the collection.
7. A generous upper bound *hi* (say 10^{18} instead of 10^9) can never make the answer wrong, only slower.

EXERCISE 6.5 Design before you code: machines

A factory has n machines. Machine i finishes one product every $t[i]$ minutes, and all machines work at the same time. What is the least number of minutes T after which at least M products are finished? Limits: $n \le 2 \cdot 10^5$, $t[i] \le 10^9$, $M \le 10^9$. This problem is not in the plan.

Write the predicate ok(T), prove in one line that it is monotone, choose *lo* and *hi* (with a reason why ok(*hi*) is true), give the cost of one check and of the whole search, and name the overflow risk inside the check.

Run your search on t = [3, 2, 5] and M = 10. Write each (lo, hi, mid, products) row.

EXERCISE 6.6 Find the bug

This method is meant to compute the largest possible smallest piece sum (*Theory Listing 6.6*) but sometimes never returns. It compiles. Find the smallest input on which it hangs, explain the cause with the values of *lo*, *hi* and *mid*, and fix it with one change.

```
static long maxMinPieceBuggy(int[] a, int k) {
    long lo = 0, hi = 0;
    for (int x : a) hi += x;
    while (lo < hi) {
```

```
        long mid = lo + (hi - lo) / 2;
        if (piecesAtLeast(a, mid) >= k) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}

static int piecesAtLeast(int[] a, long s) {
    int pieces = 0;
    long cur = 0;
    for (int x : a) {
        cur += x;
        if (cur >= s) {
            pieces++;
            cur = 0;
        }
    }
    return pieces;
}
```

EXERCISE 6.7 **The k-th smallest by counting**

Consider the 4×5 multiplication table (entries $i \cdot j$ for $1 \le i \le 4$, $1 \le j \le 5$). Find its 11th smallest entry with *Theory Listing 6.9* without listing the table.

x	5	6	7	8	10
count(x) = Σ min(5, ⌊x/i⌋)					

lo	hi	mid	count(mid)	decision
1	20			

count(6) and count(7) are equal. Why does the search return 6 and not 7?

EXERCISE 6.8 Complexity and limits

For each situation, give the bounds *lo* and *hi*, whether they must be `long`, and the number of iterations of the search.

1. LeetCode 875: piles up to 10^9, at most 10^4 piles.
2. LeetCode 1011: at most $5 \cdot 10^4$ packages of weight at most 500.
3. LeetCode 410: at most 1000 values, each at most 10^6.
4. A real answer in $[0, 10^9]$ needed to within 10^{-6}.

EXERCISE 6.9 Write it from memory

Without looking, write `firstTrue` and `lastTrue` (*Theory Listing 6.4*) with their requirements as comments. Mark the line where the two differ in rounding, and say why.

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

6.P1 LC 34 · Find First and Last Position of Element in Sorted Array MEDIUM CORE — 40 min

`leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P1*

6.P2 LC 875 · Koko Eating Bananas MEDIUM CORE — 40 min

`leetcode.com/problems/koko-eating-bananas` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P2*

6.P3 LC 1011 · Capacity To Ship Packages Within D Days MEDIUM CORE — 40 min

`leetcode.com/problems/capacity-to-ship-packages-within-d-days` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P3*

6.P4 LC 410 · Split Array Largest Sum HARD CORE — 60 min

`leetcode.com/problems/split-array-largest-sum` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P4*

6.P5 LC 1552 · Magnetic Force Between Two Balls MEDIUM BONUS 40 min

leetcode.com/problems/magnetic-force-between-two-balls · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P5*

6.P6 LC 2226 · Maximum Candies Allocated to K Children MEDIUM BONUS 40 min

leetcode.com/problems/maximum-candies-allocated-to-k-children · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P6*

6.P7 LC 719 · Find K-th Smallest Pair Distance HARD BONUS 60 min

leetcode.com/problems/find-k-th-smallest-pair-distance · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P7*

Re-solve from three days ago

Pick one problem from Day 3 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 3.P1 LC 3 · Longest Substring Without Repeating Characters ☐ 3.P2 LC 1004 · Max Consecutive Ones III

☐ 3.P3 LC 424 · Longest Repeating Character Replacement ☐ 3.P4 LC 15 · 3Sum

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 7, Monotonic stacks.

DAY 7 OF 29 · FRIDAY 13 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

7

Monotonic Stacks

Next or previous greater/smaller element for every index in O(n), and the contribution trick.

Read first · Theory Chapter 7
Patterns · Monotonic stack

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 7; type each new listing once and run it	40 min
☐	7.P1 LC 739 · Daily Temperatures	40 min
☐	7.P2 LC 503 · Next Greater Element II	40 min
☐	7.P3 LC 84 · Largest Rectangle in Histogram	60 min
☐	7.P4 LC 907 · Sum of Subarray Minimums	40 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 7.P5, 7.P6, 7.P7*	*60 min*
	Core total (estimate)	**4.1 h**

NOTES FOR THE DAY

- Keep indices whose answer is unknown on a stack. A bigger value answers all smaller ones on top.
- Contribution: a[i] is the minimum of (i − left) × (right − i) subarrays. Use < on one side and ≤ on the other.
- Use `ArrayDeque` or an `int[]` with a top pointer, never `Stack`.

Why it matters for Infosys. "Next greater element" and "remove K digits for the smallest ticket price" both appear in Infosys reports.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Binary search: lower and upper bound	Ch. 6	1 day ago	☐ Again ☐ Shaky ☐ Solid
Binary search on the answer	Ch. 6	1 day ago	☐ Again ☐ Shaky ☐ Solid
Intervals: sort, then sweep	Ch. 4	3 days ago	☐ Again ☐ Shaky ☐ Solid

Exercises

Work these after reading Theory Chapter 7. Keep the stack contents written out at every step; most stack bugs show up as a wrong stack after one particular step, and a hand trace finds that step.

EXERCISE 7.1 Fill the gaps

Complete the next-greater template. Then compare with *Theory Listing 7.1*.

```
static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    ______________________;                          // (1) the answer when none exists
    int[] st = new int[n];
    int top = 0;
    for (int i = 0; i < n; i++) {
        while (__________ && ________________________)   // (2) stack not empty, (3) top is smaller
            ______________________;                  // (4) pop it and record its answer
        ______________________;                      // (5) push i
    }
    return res;
}
```

What must change in line (3) to get the next greater or equal element? And to get the next smaller element?

EXERCISE 7.2 Trace the next-greater stack

Trace *Theory Listing 7.1* on a = [4, 2, 1, 3, 5, 2, 6]. Write the indices popped at each step and the stack (as indices) after the push.

i	a[i]	popped (their answer is i)	stack after
0	4		
1	2		
2	1		
3	3		
4	5		
5	2		
6	6		

The result array, and the total number of pops. Why can the total never exceed 7 here?

EXERCISE 7.3 A faded contribution table

Compute the sum of subarray minimums of a = [3, 1, 2, 2, 1] with *Theory Theorem 7.6*: left = previous strictly smaller index, right = next smaller-or-equal index. The first two rows are done.

i	a[i]	left	right	(i − left) · (right − i)	a[i] · count
0	3	−1	1	1 · 1 = 1	3
1	1	−1	4	2 · 3 = 6	6
2	2				
3	2				
4	1				

Total, and the check on the counts. Index 1 holds the value 1 but owns only six subarrays, although more subarrays contain it. Which subarray containing index 1 does it not own, and who owns it?

EXERCISE 7.4 A histogram by hand

For the bars h = [2, 4, 3, 5, 1, 3], fill in the previous and next strictly lower bar of each bar, the width of its rectangle and its area (*Theory Theorem 7.4*).

bar i	h[i]	L(i)	R(i)	width R − L − 1	area
0	2				
1	4				
2	3				
3	5				
4	1				
5	3				

Draw the histogram and shade the largest rectangle.

EXERCISE 7.5 Concept check

Answer true or false, and give a one-line reason.

1. One iteration of the `for` loop in *Theory Listing 7.1* can perform up to $n - 1$ pops, so the method is $O(n^2)$ in the worst case.
2. For the previous strictly smaller element, pop while `a[top] >= a[i]` and read the top after popping.
3. In the contribution formula, using strict comparisons on both sides over-counts subarrays whose minimum occurs more than once.
4. The sum of subarray ranges equals the sum of subarray maximums minus the sum of subarray minimums.
5. For a circular array, it is wrong to push indices during the second lap.
6. In *Theory Listing 7.7*, if k deletions remain after the scan, they are taken from the front of the stack.

EXERCISE 7.6 Design before you code: price spans

Each day has a price. The **span** of day i is the number of consecutive days ending at day i (including it) whose price is $\leq$ the price of day i. This problem is not in the plan.

Which variant of *Theory Table 7.1* gives the span? Write the pop condition, the formula for the span from what is left on the stack, and the complexity.

Compute the spans for the prices [30, 25, 27, 27, 40, 22, 26].

EXERCISE 7.7 Find the bug

This version of the contribution sum compiles and passes every test with distinct values. Find the smallest positive input on which it is wrong, give both answers, and fix it with one character.

```
static long sumMinsBuggy(int[] a) {
    int n = a.length;
    long total = 0;
    for (int i = 0; i < n; i++) {
        int l = i - 1;
        while (l >= 0 && a[l] >= a[i]) l--;          // previous strictly smaller
        int r = i + 1;
        while (r < n && a[r] >= a[i]) r++;           // next strictly smaller
        total += (long) a[i] * (i - l) * (r - i);
    }
    return total;
}
```

EXERCISE 7.8 Delete three digits

Run *Theory Listing 7.7* on the digits “3141592” with $k = 3$. Write what is popped at each step, the stack after the step and the deletions left.

digit	popped	stack after	k left
3			
1			
4			
1			
5			
9			
2			

The result. Why is deleting the 9 better than deleting the 5?

EXERCISE 7.9 **Write it from memory**

Write the two boundary scans of *Theory Listing 7.5* with `int[]` stacks instead of `ArrayDeque`, and the final sum. Mark which comparison is strict.

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

7.P1 **LC 739 · Daily Temperatures** MEDIUM CORE **40 min**

`leetcode.com/problems/daily-temperatures` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 7.P1*

7.P2 LC 503 · Next Greater Element II MEDIUM CORE — 40 min

leetcode.com/problems/next-greater-element-ii · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 7.P2

7.P3 LC 84 · Largest Rectangle in Histogram HARD CORE — 60 min

leetcode.com/problems/largest-rectangle-in-histogram · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 7.P3

7.P4 LC 907 · Sum of Subarray Minimums MEDIUM CORE — 40 min

leetcode.com/problems/sum-of-subarray-minimums · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 7.P4

7.P5 LC 402 · Remove K Digits MEDIUM BONUS — 40 min

leetcode.com/problems/remove-k-digits · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 7.P5

7.P6 **LC 316 · Remove Duplicate Letters** MEDIUM BONUS **40 min**

leetcode.com/problems/remove-duplicate-letters · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 7.P6*

7.P7 **LC 2104 · Sum of Subarray Ranges** MEDIUM BONUS **40 min**

leetcode.com/problems/sum-of-subarray-ranges · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 7.P7*

Re-solve from three days ago

Pick one problem from Day 4 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 4.P1 LC 56 · Merge Intervals ☐ 4.P2 LC 435 · Non-overlapping Intervals
☐ 4.P3 LC 2406 · Divide Intervals Into Minimum Number of Groups ☐ 4.P4 LC 179 · Largest Number

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 8, Bits and XOR.

DAY 8 OF 29 · SATURDAY 14 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

8

Bits and XOR

Each bit is independent in a sum of XORs. A trie answers pair questions; a basis answers subset questions.

Read first · Theory Chapter 8
Patterns · Bit tricks and XOR, Binary trie for XOR, XOR basis (maximum subset XOR)

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 8; type each new listing once and run it	40 min
☐	8.P1 LC 136 · Single Number	20 min
☐	8.P2 LC 1442 · Count Triplets That Can Form Two Arrays of Equal XOR	40 min
☐	8.P3 LC 421 · Maximum XOR of Two Numbers in an Array	40 min
☐	8.P4 LC 1803 · Count Pairs With XOR in a Range	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 8.P5, 8.P6, 8.P7*	*60 min*
	Core total (estimate)	**3.8 h**

NOTES FOR THE DAY

- `x & -x` is the lowest set bit; write `1L << k` when k can reach 31.
- Prefix XOR: `xor(l..r) = P[r+1] ^ P[l]`, exactly like prefix sums.
- Binary trie: walk from the top bit, choosing the opposite bit to maximise the XOR.
- XOR basis: at most one number per highest bit; the maximum subset XOR is built greedily.

Why it matters for Infosys. There are two XOR repeats: "at most N/2 elements, max XOR" (13 reports) and "X ≤ K maximising Σ X xor A[i]" (6 reports). The sample Hard problem is XOR-based too.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Monotonic stack	Ch. 7	1 day ago	☐ Again ☐ Shaky ☐ Solid
Greedy with a heap	Ch. 5	3 days ago	☐ Again ☐ Shaky ☐ Solid
Java collections toolkit	Ch. 1	7 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Fast input and output (FastScanner)	Ch. 1	7 days ago	☐ Again ☐ Shaky ☐ Solid
Linked list moves	Ch. 1	7 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

8.P1 LC 136 · Single Number EASY CORE — 20 min

leetcode.com/problems/single-number · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P1*

8.P2 LC 1442 · Count Triplets That Can Form Two Arrays of Equal XOR MEDIUM CORE — 40 min

leetcode.com/problems/count-triplets-that-can-form-two-arrays-of-equal-xor · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P2*

8.P3 LC 421 · Maximum XOR of Two Numbers in an Array MEDIUM CORE — 40 min

leetcode.com/problems/maximum-xor-of-two-numbers-in-an-array · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P3*

8.P4 LC 1803 · Count Pairs With XOR in a Range HARD CORE

60 min

leetcode.com/problems/count-pairs-with-xor-in-a-range · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 8.P4

8.P5 LC 2429 · Minimize XOR MEDIUM BONUS

40 min

leetcode.com/problems/minimize-xor · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 8.P5

8.P6 GfG · Maximum Subset XOR GFG BONUS

40 min

geeksforgeeks.org/problems/maximum-subset-xor/1 · GeeksforGeeks

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 8.P6

8.P7 LC 1707 · Maximum XOR With an Element From Array HARD BONUS

60 min

leetcode.com/problems/maximum-xor-with-an-element-from-array · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 8.P7

Re-solve from three days ago

Pick one problem from Day 5 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 5.P1 LC 502 · IPO ☐ 5.P2 LC 1642 · Furthest Building You Can Reach

☐ 5.P3 LC 1526 · Minimum Number of Increments on Subarrays to Form a Target Array ☐ 5.P4 LC 1046 · Last Stone Weight

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 9, Mock 1 + review.

DAY 9 OF 29 · SUNDAY 15 NOVEMBER 2026 · MOCK DAY · PHASE 1 · Q1/Q2 TOOLKIT

9

Mock 1 + review

3 hours on Phase 1 topics at Q1/Q2/Q3 difficulty, then a full review.

Read first · Theory Chapter 9
Patterns · Brute force first, then stress test

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	9.P1 CF 1343C · Alternating Subsequence	30 min
☐	9.P2 CF 1475D · Cleaning the Phone	60 min
☐	9.P3 CF 1251D · Salary Changing	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
	Core total (estimate)	**4.0 h**

NOTES FOR THE DAY

- Read the mock rules. Open each problem, then type your program on the Codeforces *Submit* page (language: Java). Test with the samples on *Custom test* first.
- Before the mock, read the handbook card "Brute force first": it's how you collect partial marks.
- After the mock: enter your scores in the Mocks section, read the editorials, re-code every miss and log each cause.

Why it matters for Infosys. This is your first measurement. Compare it with the SP checkpoints.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Bit tricks and XOR	Ch. 8	1 day ago	☐ Again ☐ Shaky ☐ Solid
Binary trie for XOR	Ch. 8	1 day ago	☐ Again ☐ Shaky ☐ Solid
XOR basis (maximum subset XOR)	Ch. 8	1 day ago	☐ Again ☐ Shaky ☐ Solid
Binary search: lower and upper bound	Ch. 6	3 days ago	☐ Again ☐ Shaky ☐ Solid
Binary search on the answer	Ch. 6	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Prefix sums + HashMap	Ch. 2	7 days ago	☐ Again ☐ Shaky ☐ Solid
Difference array (many range adds)	Ch. 2	7 days ago	☐ Again ☐ Shaky ☐ Solid

Mock exam: Mock 1 · Phase 1 topics

Q1 ≈ 1200, Q2 ≈ 1800, Q3 ≈ 1900 (Codeforces ratings). Weights 20/30/50. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- ☐ **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- ☐ **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- ☐ **By minute 150** every problem has a submission, even if it is only a brute force.
- ☐ **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

Planning grid (first 15 minutes)

Q	Limits	Idea	Complexity	Brute-force fallback	Order
Q1					
Q2					
Q3					

Score sheet

Q	Problem	Rating	Weight	Start	End	% tests	Marks
Q1	**CF 1343C** Alternating Subsequence `codeforces.com/problemset/problem/1343/C`	1200	20				
Q2	**CF 1475D** Cleaning the Phone `codeforces.com/problemset/problem/1475/D`	1800	30				
Q3	**CF 1251D** Salary Changing `codeforces.com/problemset/problem/1251/D`	1900	50				
	Weighted score = Σ weight × % tests passed						

SP checkpoint. Q1 fully solved; Q2 at least half the tests.

Post-mortem (about 60 minutes after the mock)

Q	What went wrong	Cause	Rule that would have prevented it
Q1		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q2		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q3		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	

Re-solve from three days ago

Pick one problem from Day 6 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- [ ] 6.P1 LC 34 · Find First and Last Position of Element in Sorted Array
- [ ] 6.P2 LC 875 · Koko Eating Bananas
- [ ] 6.P3 LC 1011 · Capacity To Ship Packages Within D Days
- [ ] 6.P4 LC 410 · Split Array Largest Sum

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 10, Number theory toolkit.

DAY 10 OF 29 · MONDAY 16 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

10

Number Theory for Programmers

gcd, sieves, fast power, inverses, nCr, and the square-free kernel.

Read first · Theory Chapter 10
Patterns · gcd, fast power, inverse, nCr, Sieve, prime factors, square-free kernel

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 10; type each new listing once and run it	40 min
☐	10.P1 LC 204 · Count Primes	40 min
☐	10.P2 LC 1922 · Count Good Numbers	40 min
☐	10.P3 LC 2447 · Number of Subarrays With GCD Equal to K	40 min
☐	10.P4 LC 2862 · Maximum Element-Sum of a Complete Subset of Indices	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 10.P5, 10.P6, 10.P7*	*60 min*
	Core total (estimate)	**4.1 h**

NOTES FOR THE DAY

- `lcm = a / gcd(a, b) * b`: divide first so it doesn't overflow.
- The smallest-prime-factor sieve factorises any $x \le 10^7$ in O(log x).
- Fermat inverse `a^(MOD-2)` when MOD is prime; precompute factorials for O(1) nCr. A non-prime modulus (Infosys has used 10000) needs Pascal's triangle.
- x·y is a perfect square ⇔ kernel(x) = kernel(y), where the kernel keeps the primes with an odd exponent.

Why it matters for Infosys. The sample Easy (perfect-square products) and Medium (gcd exactly p) are both number theory at their core.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Brute force first, then stress test	Ch. 9	1 day ago	☐ Again ☐ Shaky ☐ Solid
Monotonic stack	Ch. 7	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Two pointers on a sorted array	Ch. 3	7 days ago	☐ Again ☐ Shaky ☐ Solid
Sliding window (variable size)	Ch. 3	7 days ago	☐ Again ☐ Shaky ☐ Solid
Window maximum (monotonic deque)	Ch. 3	7 days ago	☐ Again ☐ Shaky ☐ Solid

Exercises

Work these after reading Theory Chapter 10 and before the problem worksheets. The first four are traces of the chapter's algorithms, which you should do with pencil and paper only: every intermediate number is small enough. The rest ask you to decide, explain and debug.

EXERCISE 10.1 Fill the gaps

Complete fast power and the factorial precomputation from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 10.3* and *Theory Listing 10.5*.

```
static long power(long base, long exp, long m) {
    long r = _____________;                // (1) so that m = 1 gives 0
    base %= m;
    if (base < 0) base += m;
    while (exp > 0) {
        if (_____________) r = r * base % m; // (2) the lowest bit of exp is set
        ______________________________;       // (3) square the base
        ______________________________;       // (4) drop the lowest bit
    }
    return r;
}

static void initFactorials(int n) {
    fact = new long[n + 1];
    invFact = new long[n + 1];
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[n] = ______________________________;                 // (5) one Fermat inverse
    for (int i = n; i > 0; i--) invFact[i - 1] = _____________;   // (6)
}
```

Why is line (6) correct? Write the identity it uses.

EXERCISE 10.2 Trace Euclid's algorithm

Run *Theory Listing 10.1* on $a = 2024$, $b = 1717$. Each row is one iteration of the loop. Then answer the questions below.

iteration	a	b	q = ⌊a/b⌋	r = a mod b
1	2024	1717		
2				

iteration	a	b	q = ⌊a/b⌋	r = a mod b
3				
4				
5				
6				
7				
8				

gcd(2024, 1717) = ? How many iterations did the loop make? What is lcm(2024, 1717), computed as `a / gcd * b`?

EXERCISE 10.3 **A faded trace: the extended algorithm**

Theory Listing 10.2 returns from the deepest call first. Use the quotients q from Exercise 10.2. Each row computes $x = y'$ and $y = x' - q \cdot y'$ from the row above it (x', y'). The first four rows are done. Fill in the rest and check the last row: $2024x + 1717y$ must equal 1.

call (a, b)	q	x	y	check a·x + b·y
(1, 0)	—	1	0	1
(2, 1)	2	0	1	1
(11, 2)	5	1	−5	1
(57, 11)	5	−5	26	1
(125, 57)	2			
(182, 125)	1			
(307, 182)	1			
(1717, 307)	5			
(2024, 1717)	1			

Use the last row to write the inverse of 1717 modulo 2024 as a number in 0..2023.

EXERCISE 10.4 Trace fast power

Compute 7^{45} mod 1000 with *Theory Listing 10.3*. First write 45 in binary. Each row is one iteration: the value of exp at the start, its lowest bit, and then *r*, base and exp after the iteration.

exp at start	bit	r after	base after	exp after
45				

45 in binary = _____. The answer 7^{45} mod 1000 = _____. How many multiplications would the naive loop use?

EXERCISE 10.5 Smallest prime factors and kernels by hand

Fill in `spf[x]` for $2 \le x \le 40$, as *Theory Listing 10.9* would. The cell in row *t* and column +*u* is $x = t + u$. Circle the primes.

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9
0	—	—								
10										
20										
30										
40		—	—	—	—	—	—	—	—	—

Now factorise 756 and 84 with the table, writing the chain of divisions as in the theory chapter. Give each one's factorisation, number of divisors and square-free kernel. Is $756 \cdot 84$ a perfect square? Is $756 \cdot 1176$ one ($1176 = 2^3 \cdot 3 \cdot 7^2$)? Decide from the kernels, without multiplying.

EXERCISE 10.6 Concept check

Answer true or false, and give a one-line reason or a counterexample.

1. If b divides a, then $(a / b) \bmod m = (a \bmod m) / (b \bmod m)$.
2. If $\gcd(a, 10\,000) = 1$, then $a^{9998} \bmod 10\,000$ is the inverse of a modulo 10 000.
3. *Theory Listing 10.1* returns $\gcd(0, 0) = 0$ and $\gcd(0, 12) = 12$.
4. The sum of two residues modulo $10^9 + 7$ always fits in an `int`.
5. $x \cdot y$ is a perfect square exactly when x and y have the same set of prime factors.
6. The sieve of Eratosthenes up to n costs $O(n \log n)$, because of the harmonic series.
7. An `int[]` SPF sieve up to 10^7 needs about 40 MB.

EXERCISE 10.7 Design before you code

A new problem, not in the plan: given $n \le 10^5$ values $1 \le a[i] \le 10^7$, count the pairs $i < j$ such that $a[i] \cdot a[j]$ is a perfect square. Write the key idea, the data structures, the formula for the answer, and the time and memory cost. Then compute the answer by hand for $a = [2, 8, 3, 12, 18, 5, 50, 27, 1, 4]$.

EXERCISE 10.8 **Find the bug**

This fast power was typed in a hurry. It compiles and works on small inputs such as `powerBuggy(3, 5, 1000)` = 243, but it fails modulo $10^9 + 7$. Find the smallest kind of input that breaks it, give one failing call with the wrong and the right result, and fix it.

```
static int powerBuggy(int base, int exp, int m) {
    int r = 1;
    base %= m;
    while (exp > 0) {
        if ((exp & 1) == 1) r = r * base % m;
        base = base * base % m;
        exp >>= 1;
    }
    return r;
}
```

EXERCISE 10.9 **Complexity and limits**

For each task, name the method from Theory Chapter 10, its time and memory, and one trap.

1. Answer 10^5 queries $C(n, r)$ mod $10^9 + 7$ with $n \le 10^6$.
2. Compute $C(n, r)$ mod 10 000 for all $n \le 1000$.
3. Decide for each of 10^5 numbers up to 10^7 whether it is prime.
4. Factorise one number up to 10^{12}.
5. Compute a^e mod $10^9 + 7$ for e up to 10^{18}.
6. Compute the square-free kernels of 10^5 numbers up to 10^9.

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time

is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

10.P1 **LC 204 · Count Primes** MEDIUM CORE — **40 min**

leetcode.com/problems/count-primes · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P1*

10.P2 **LC 1922 · Count Good Numbers** MEDIUM CORE — **40 min**

leetcode.com/problems/count-good-numbers · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P2*

10.P3 **LC 2447 · Number of Subarrays With GCD Equal to K** MEDIUM CORE — **40 min**

leetcode.com/problems/number-of-subarrays-with-gcd-equal-to-k · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P3*

10.P4 **LC 2862 · Maximum Element-Sum of a Complete Subset of Indices** HARD CORE — **60 min**

leetcode.com/problems/maximum-element-sum-of-a-complete-subset-of-indices · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P4*

10.P5 LC 1819 · Number of Different Subsequences GCDs HARD BONUS

60 min

leetcode.com/problems/number-of-different-subsequences-gcds · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 10.P5

10.P6 LC 2572 · Count the Number of Square-Free Subsets MEDIUM BONUS

40 min

leetcode.com/problems/count-the-number-of-square-free-subsets · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 10.P6

10.P7 CF 1497E1 · Square-Free Division (easy version) CF 1700 BONUS

45 min

codeforces.com/problemset/problem/1497/E1 · Codeforces

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 10.P7

Re-solve from three days ago

Pick one problem from Day 7 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 7.P1 LC 739 · Daily Temperatures ☐ 7.P2 LC 503 · Next Greater Element II

☐ 7.P3 LC 84 · Largest Rectangle in Histogram ☐ 7.P4 LC 907 · Sum of Subarray Minimums

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 11, DP I: states, transitions, knapsack.

DAY 11 OF 29 · TUESDAY 17 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

11

DP I: states, transitions, knapsack

Define the state in words before writing any code.

Read first · Theory Chapter 11
Patterns · DP recipe: state, transition, base, answer, Knapsack family

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 11; type each new listing once and run it	40 min
☐	11.P1 LC 198 · House Robber	40 min
☐	11.P2 LC 322 · Coin Change	40 min
☐	11.P3 LC 518 · Coin Change II	40 min
☐	11.P4 LC 416 · Partition Equal Subset Sum	40 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 11.P5, 11.P6, 11.P7*	*60 min*
	Core total (estimate)	**3.8 h**

NOTES FOR THE DAY

- Write it down first: *dp[i]* = …, the transition, the base case, and where the answer is.
- 0/1 knapsack: loop capacity *downwards*. Unbounded: loop it upwards.
- Counting combinations vs orderings: coins in the outer loop for combinations.
- Prefer loops over recursion in Java for deep DP.

Why it matters for Infosys. Coin change, subset sum and 0/1 knapsack are among the most-reported Infosys DP problems.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
gcd, fast power, inverse, nCr	Ch. 10	1 day ago	☐ Again ☐ Shaky ☐ Solid
Sieve, prime factors, square-free kernel	Ch. 10	1 day ago	☐ Again ☐ Shaky ☐ Solid
Bit tricks and XOR	Ch. 8	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Binary trie for XOR	Ch. 8	3 days ago	☐ Again ☐ Shaky ☐ Solid
XOR basis (maximum subset XOR)	Ch. 8	3 days ago	☐ Again ☐ Shaky ☐ Solid
Intervals: sort, then sweep	Ch. 4	7 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

11.P1 LC 198 · House Robber MEDIUM CORE — **40 min**

leetcode.com/problems/house-robber · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P1*

11.P2 LC 322 · Coin Change MEDIUM CORE — **40 min**

leetcode.com/problems/coin-change · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P2*

11.P3 LC 518 · Coin Change II MEDIUM CORE — **40 min**

leetcode.com/problems/coin-change-ii · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P3*

11.P4 **LC 416 · Partition Equal Subset Sum** MEDIUM CORE — 40 min

leetcode.com/problems/partition-equal-subset-sum · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P4*

11.P5 **LC 494 · Target Sum** MEDIUM BONUS — 40 min

leetcode.com/problems/target-sum · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P5*

11.P6 **LC 1049 · Last Stone Weight II** MEDIUM BONUS — 40 min

leetcode.com/problems/last-stone-weight-ii · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P6*

11.P7 **LC 740 · Delete and Earn** MEDIUM BONUS — 40 min

leetcode.com/problems/delete-and-earn · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P7*

Re-solve from three days ago

Pick one problem from Day 8 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 8.P1 LC 136 · Single Number ☐ 8.P2 LC 1442 · Count Triplets That Can Form Two Arrays of Equal XOR
☐ 8.P3 LC 421 · Maximum XOR of Two Numbers in an Array ☐ 8.P4 LC 1803 · Count Pairs With XOR in a Range

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 12, DP II: two-string DP.

DAY 12 OF 29 · WEDNESDAY 18 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

12

DP II: two-string DP

dp[i][j] over the prefixes of two strings.

Read first · Theory Chapter 12
Patterns · Two-string DP (LCS, edit distance)

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 12; type each new listing once and run it	40 min
☐	12.P1 LC 1143 · Longest Common Subsequence	40 min
☐	12.P2 LC 72 · Edit Distance	40 min
☐	12.P3 LC 516 · Longest Palindromic Subsequence	40 min
☐	12.P4 LC 1312 · Minimum Insertion Steps to Make a String Palindrome	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 12.P5, 12.P6, 12.P7*	*60 min*
	Core total (estimate)	**4.1 h**

NOTES FOR THE DAY

- dp[i][j] = answer for the first i characters of s and the first j characters of t.
- Longest palindromic subsequence = LCS(s, reverse(s)); fewest insertions = n − that.
- To print the answer (not just its length), walk the table back from dp[n][m].

Why it matters for Infosys. LCS and edit distance are both on the Infosys "very high" list.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
DP recipe: state, transition, base, answer	Ch. 11	1 day ago	☐ Again ☐ Shaky ☐ Solid
Knapsack family	Ch. 11	1 day ago	☐ Again ☐ Shaky ☐ Solid
Brute force first, then stress test	Ch. 9	3 days ago	☐ Again ☐ Shaky ☐ Solid
Greedy with a heap	Ch. 5	7 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

12.P1 **LC 1143 · Longest Common Subsequence** MEDIUM CORE — **40 min**

`leetcode.com/problems/longest-common-subsequence` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet — *Appendix A, hint 12.P1*

12.P2 **LC 72 · Edit Distance** MEDIUM CORE — **40 min**

`leetcode.com/problems/edit-distance` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet — *Appendix A, hint 12.P2*

12.P3 **LC 516 · Longest Palindromic Subsequence** MEDIUM CORE — **40 min**

`leetcode.com/problems/longest-palindromic-subsequence` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet — *Appendix A, hint 12.P3*

12.P4 **LC 1312 · Minimum Insertion Steps to Make a String Palindrome** HARD CORE — **60 min**

`leetcode.com/problems/minimum-insertion-steps-to-make-a-string-palindrome` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet — *Appendix A, hint 12.P4*

12.P5 **LC 115 · Distinct Subsequences** HARD BONUS — 60 min

leetcode.com/problems/distinct-subsequences · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P5*

12.P6 **LC 97 · Interleaving String** MEDIUM BONUS — 40 min

leetcode.com/problems/interleaving-string · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P6*

12.P7 **LC 1092 · Shortest Common Supersequence** HARD BONUS — 60 min

leetcode.com/problems/shortest-common-supersequence · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P7*

Re-solve from three days ago

Pick one problem from Day 9 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 9.P1 CF 1343C · Alternating Subsequence ☐ 9.P2 CF 1475D · Cleaning the Phone

☐ 9.P3 CF 1251D · Salary Changing

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 13, DP III: LIS and DP with an extra dimension.

DAY 13 OF 29 · THURSDAY 19 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

13

Dynamic Programming III: Increasing Subsequences and Extra Dimensions

LIS in O(n log n), DP over values with a Fenwick tree, and an extra state for a side condition.

Read first · Theory Chapter 13
Patterns · Longest increasing subsequence

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 13; type each new listing once and run it	40 min
☐	13.P1 LC 300 · Longest Increasing Subsequence	40 min
☐	13.P2 LC 1964 · Find the Longest Valid Obstacle Course at Each Position	60 min
☐	13.P3 LC 1671 · Minimum Number of Removals to Make Mountain Array	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 13.P4, 13.P5, 13.P6*	*60 min*
	Core total (estimate)	**4.0 h**

NOTES FOR THE DAY

- tails + lower bound gives the strict LIS; upper bound gives the non-decreasing one.
- DP over values: dp(x) = 1 + max dp over smaller values, kept in a Fenwick tree of maximums.
- A side condition with few states becomes an extra dimension. The sample Hard problem uses 1024 XOR states.

Why it matters for Infosys. Three Infosys repeats are LIS variants: LNDS with XOR ≥ M (13 reports), LIS with a bitwise condition, and mountain-array changes.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Two-string DP (LCS, edit distance)	Ch. 12	1 day ago	☐ Again ☐ Shaky ☐ Solid
gcd, fast power, inverse, nCr	Ch. 10	3 days ago	☐ Again ☐ Shaky ☐ Solid
Sieve, prime factors, square-free kernel	Ch. 10	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Binary search: lower and upper bound	Ch. 6	7 days ago	☐ Again ☐ Shaky ☐ Solid
Binary search on the answer	Ch. 6	7 days ago	☐ Again ☐ Shaky ☐ Solid

Exercises

Work these after reading Theory Chapter 13 and before the problem worksheets. The first four make you run the chapter's algorithms by hand, which is the fastest way to see why the tails array and the Fenwick tree give the right numbers. The rest ask you to decide, design and debug without a printed answer in front of you.

EXERCISE 13.1 Fill the gaps: patience sorting

Complete the template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 13.2*.

```
static int lisStrict(int[] a) {
    int[] tails = new int[a.length];
    int len = 0;
    for (int x : a) {
        int lo = 0, hi = ______;                  // (1)
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (tails[mid] ______ x) hi = mid;    // (2) strictly increasing
            else lo = mid + 1;
        }
        ______________________;                   // (3)
        ______________________;                   // (4)
    }
    return len;
}
```

Which single character changes in blank (2) for the longest non-decreasing subsequence, and why?

EXERCISE 13.2 Trace patience sorting

Run *Theory Listing 13.2* (strict) on a = [4, 10, 4, 3, 8, 9, 1, 5]. For each element write the position p returned by the binary search, the best length ending at that element ($p + 1$), and `tails` after the step.

x	position p	p + 1	tails after
4			
10			
4			
3			
8			

x	position p	p + 1	tails after
9			
1			
5			

The LIS length is _____. Write one longest increasing subsequence, and explain why the final `tails` is not one.

EXERCISE 13.3 A faded table: the mountain

For a = [2, 5, 3, 6, 4, 1, 7, 2], up[i] is the longest strictly increasing subsequence ending at i and down[i] the longest strictly decreasing subsequence starting at i. The first three columns are done. Fill in the rest, mark the indices that may be a peak (both values at least 2), and compute the fewest removals that leave a mountain.

i	0	1	2	3	4	5	6	7
a[i]	2	5	3	6	4	1	7	2
up[i]	1	2	2					
down[i]	2	3	2					
peak allowed?	no	yes	yes					
up + down − 1	—	4	3					

Longest mountain = _____, so the fewest removals = _____. Write one longest mountain subsequence.

EXERCISE 13.4 The Fenwick tree of maxima by hand

Compress a = [30, 10, 50, 10, 20, 40] to ranks 1..V, then run *Theory Listing 13.6* (strict). For each element write its rank, the query result `prefixMax(fw, rank - 1)`, the new length `cur`, and the array `fw[1..V]` after `raise`. Remember that `raise` at rank r writes nodes r, r + lowbit(r), … up to V.

a[i]	rank	prefixMax(rank − 1)	cur	fw[1..V] after
30				
10				
50				

a[i]	rank	prefixMax(rank − 1)	cur	fw[1..V] after
10				
20				
40				

Why does the second 10 get `cur` = 1 even though a 10 was seen before?

EXERCISE 13.5 Concept check

Answer true or false, and give a one-line reason.

1. After patience sorting finishes, `tails` is a longest increasing subsequence of a.
2. For the longest non-decreasing subsequence, each element replaces the first tail strictly greater than it.
3. In the mountain formula, an index with up[i] = 1 can be the peak if down[i] is large.
4. A Fenwick tree of maxima can process an update that lowers a stored value.
5. For envelopes that nest when both sides are strictly smaller, envelopes of equal width are sorted by height descending.
6. Adding a remainder modulo K as an extra state multiplies the work of the DP by about K.

EXERCISE 13.6 Design before you code: towers of blocks

You have $n \le 10^5$ blocks, each with a width w, a height h (both up to 10^9) and a value $v \ge 1$. A tower is a set of blocks that can be ordered so that both w and h strictly increase from top to bottom. Find the largest total value of a tower. Write the sort order (including ties), what the Fenwick tree stores and is indexed by, the strict or non-strict query, and the complexity. Then compute the answer by hand for the blocks (w, h, v) = (1, 3, 4), (2, 2, 5), (2, 4, 3), (3, 5, 6), (4, 1, 2), (4, 6, 7).

EXERCISE 13.7 Find the bug

This method should return the length of the longest **strictly** increasing subsequence of ranks in $1..V$. It compiles and passes the sample [3, 1, 2, 4]. Find an input where it fails, say what it returns there, and fix it with one change.

```
static int lisRanks(int[] rank, int V) {
    int[] fw = new int[V + 1];
    int best = 0;
    for (int x : rank) {
        int cur = 1;
        for (int i = x; i > 0; i -= i & -i) cur = Math.max(cur, fw[i] + 1);
        for (int i = x; i <= V; i += i & -i) fw[i] = Math.max(fw[i], cur);
        best = Math.max(best, cur);
    }
    return best;
}
```

EXERCISE 13.8 Complexity and limits

For each task, choose the method from Theory Chapter 13 (or Chapter 9's brute force) and justify it with an operation count.

1. $n \le 3000$: the longest subsequence in which each element is a multiple of the previous one (values may repeat).
2. $n \le 2 \cdot 10^5$, values up to 10^9: the LIS length.
3. $n \le 2 \cdot 10^5$: the largest total of a strictly increasing subsequence, values up to 10^9.
4. $N \le 1000$, values at most N: the longest non-decreasing subsequence whose XOR is at least M (the sample-paper Hard problem; give the state and the cost, not the code).
5. $n \le 20$, any condition on the subsequence at all.

EXERCISE 13.9 **Write it from memory**

Close the theory book. Write *Theory Listing 13.4* (`lengthEndingAt` with the `strict` flag) from memory, then check it against the listing line by line.

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 3 core problems, 3 bonus.

13.P1 LC 300 · Longest Increasing Subsequence MEDIUM CORE — 40 min

leetcode.com/problems/longest-increasing-subsequence · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P1*

13.P2 LC 1964 · Find the Longest Valid Obstacle Course at Each Position HARD CORE — 60 min

leetcode.com/problems/find-the-longest-valid-obstacle-course-at-each-position · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P2*

13.P3 LC 1671 · Minimum Number of Removals to Make Mountain Array HARD CORE — 60 min

leetcode.com/problems/minimum-number-of-removals-to-make-mountain-array · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P3*

13.P4 LC 2370 · Longest Ideal Subsequence MEDIUM BONUS — 40 min

leetcode.com/problems/longest-ideal-subsequence · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P4*

13.P5 **LC 354 · Russian Doll Envelopes** HARD BONUS — 60 min

leetcode.com/problems/russian-doll-envelopes · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P5*

13.P6 **LC 673 · Number of Longest Increasing Subsequence** MEDIUM BONUS — 40 min

leetcode.com/problems/number-of-longest-increasing-subsequence · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P6*

Re-solve from three days ago

Pick one problem from Day 10 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 10.P1 LC 204 · Count Primes ☐ 10.P2 LC 1922 · Count Good Numbers
☐ 10.P3 LC 2447 · Number of Subarrays With GCD Equal to K
☐ 10.P4 LC 2862 · Maximum Element-Sum of a Complete Subset of Indices

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 14, DP IV: divisor, interval and game DP.

DAY 14 OF 29 · FRIDAY 20 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

14

DP IV: divisor, interval and game DP

Loops over multiples, dp[l][r] by length, and "my score minus yours" for two-player games.

Read first · Theory Chapter 14
Patterns · Interval DP and game DP

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 14; type each new listing once and run it	40 min
☐	14.P1 LC 368 · Largest Divisible Subset	40 min
☐	14.P2 LC 486 · Predict the Winner	40 min
☐	14.P3 LC 1547 · Minimum Cost to Cut a Stick	60 min
☐	14.P4 LC 1690 · Stone Game VII	40 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 14.P5, 14.P6, 14.P7*	*60 min*
	Core total (estimate)	**4.1 h**

NOTES FOR THE DAY

- Loop over multiples: `for (m = 2 * v; m <= N; m += v)` costs N·ln N in total.
- Interval DP: `dp[l][r]`, filled by increasing length; $O(n^3)$ for $n \le 500$.
- Two players taking from the ends: `dp[l][r]` = best (my score − opponent's score).

Why it matters for Infosys. "Count arrays where each element divides the next" (13 reports) is divisor DP; the hard version is the bonus LC 2338.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Longest increasing subsequence	Ch. 13	1 day ago	☐ Again ☐ Shaky ☐ Solid
DP recipe: state, transition, base, answer	Ch. 11	3 days ago	☐ Again ☐ Shaky ☐ Solid
Knapsack family	Ch. 11	3 days ago	☐ Again ☐ Shaky ☐ Solid
Monotonic stack	Ch. 7	7 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

14.P1 LC 368 · Largest Divisible Subset MEDIUM CORE — **40 min**

leetcode.com/problems/largest-divisible-subset · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P1*

14.P2 LC 486 · Predict the Winner MEDIUM CORE — **40 min**

leetcode.com/problems/predict-the-winner · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P2*

14.P3 LC 1547 · Minimum Cost to Cut a Stick HARD CORE — **60 min**

leetcode.com/problems/minimum-cost-to-cut-a-stick · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P3*

14.P4 LC 1690 · Stone Game VII MEDIUM CORE — **40 min**

leetcode.com/problems/stone-game-vii · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P4*

14.P5 **LC 2338 · Count the Number of Ideal Arrays** HARD BONUS — **60 min**

leetcode.com/problems/count-the-number-of-ideal-arrays · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P5*

14.P6 **LC 312 · Burst Balloons** HARD BONUS — **60 min**

leetcode.com/problems/burst-balloons · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P6*

14.P7 **LC 664 · Strange Printer** HARD BONUS — **60 min**

leetcode.com/problems/strange-printer · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P7*

Re-solve from three days ago

Pick one problem from Day 11 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 11.P1 LC 198 · House Robber ☐ 11.P2 LC 322 · Coin Change ☐ 11.P3 LC 518 · Coin Change II
☐ 11.P4 LC 416 · Partition Equal Subset Sum

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 15, Graphs I: BFS, DFS, topological order.

DAY 15 OF 29 · SATURDAY 21 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

15

Graphs I: BFS, DFS, topological order

Shortest paths by edges, grids, multi-source BFS, Kahn's algorithm and DP on a DAG.

Read first · Theory Chapter 15
Patterns · BFS: fewest edges, grids, many sources, DFS without recursion, components, Topological order (Kahn) and DAG DP, 0-1 BFS

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 15; type each new listing once and run it	40 min
☐	15.P1 LC 200 · Number of Islands	40 min
☐	15.P2 LC 994 · Rotting Oranges	40 min
☐	15.P3 LC 207 · Course Schedule	40 min
☐	15.P4 LC 1091 · Shortest Path in Binary Matrix	40 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 15.P5, 15.P6, 15.P7*	*60 min*
	Core total (estimate)	**3.8 h**

NOTES FOR THE DAY

- BFS finds the fewest edges. For several sources, push them all first with distance 0.
- Mark a node visited when you push it, not when you pop it.
- Kahn's algorithm: repeatedly remove nodes with indegree 0. Leftover nodes mean a cycle.
- Costs of only 0 and 1: a deque replaces the heap (0 → front, 1 → back).

Why it matters for Infosys. Recent reports describe the hard question as "shortest path with a twist".

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Interval DP and game DP	Ch. 14	1 day ago	☐ Again ☐ Shaky ☐ Solid
Two-string DP (LCS, edit distance)	Ch. 12	3 days ago	☐ Again ☐ Shaky ☐ Solid
Bit tricks and XOR	Ch. 8	7 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Binary trie for XOR	Ch. 8	7 days ago	☐ Again ☐ Shaky ☐ Solid
XOR basis (maximum subset XOR)	Ch. 8	7 days ago	☐ Again ☐ Shaky ☐ Solid
Java collections toolkit	Ch. 1	14 days ago	☐ Again ☐ Shaky ☐ Solid
Fast input and output (FastScanner)	Ch. 1	14 days ago	☐ Again ☐ Shaky ☐ Solid
Linked list moves	Ch. 1	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

15.P1 LC 200 · Number of Islands MEDIUM CORE — **40 min**

`leetcode.com/problems/number-of-islands` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P1*

15.P2 LC 994 · Rotting Oranges MEDIUM CORE — **40 min**

`leetcode.com/problems/rotting-oranges` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P2*

15.P3 LC 207 · Course Schedule MEDIUM CORE — **40 min**

`leetcode.com/problems/course-schedule` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P3*

15.P4 **LC 1091 · Shortest Path in Binary Matrix** MEDIUM CORE — **40 min**

leetcode.com/problems/shortest-path-in-binary-matrix · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P4*

15.P5 **LC 785 · Is Graph Bipartite?** MEDIUM BONUS — **40 min**

leetcode.com/problems/is-graph-bipartite · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P5*

15.P6 **LC 1368 · Minimum Cost to Make at Least One Valid Path in a Grid** HARD BONUS — **60 min**

leetcode.com/problems/minimum-cost-to-make-at-least-one-valid-path-in-a-grid · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P6*

15.P7 **LC 2050 · Parallel Courses III** HARD BONUS — **60 min**

leetcode.com/problems/parallel-courses-iii · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P7*

Re-solve from three days ago

Pick one problem from Day 12 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- [ ] 12.P1 LC 1143 · Longest Common Subsequence
- [ ] 12.P2 LC 72 · Edit Distance
- [ ] 12.P3 LC 516 · Longest Palindromic Subsequence
- [ ] 12.P4 LC 1312 · Minimum Insertion Steps to Make a String Palindrome

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 16, Mock 2 + review.

DAY 16 OF 29 · SUNDAY 22 NOVEMBER 2026 · MOCK DAY · PHASE 2 · NUMBER THEORY AND DP

16

Mock 2 + review

3 hours with number theory and DP added, then a full review.

Read first · Theory Chapter 16
Patterns · —

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	16.P1 CF 1367C · Social Distance	30 min
☐	16.P2 CF 1373D · Maximum Sum on Even Positions	45 min
☐	16.P3 CF 1475G · Strange Beauty	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
	Core total (estimate)	**4.0 h**

NOTES FOR THE DAY

- Same rules as Mock 1. Use Codeforces *Custom test* to run the samples before submitting.
- After the mock: scores, editorials, re-code, error log.
- Compare with the SP checkpoints. Below them? Drop next week's bonus problems and redo DP core problems from Days 11–14.

Why it matters for Infosys. Mock 2 is the checkpoint for Q1 + Q2 fully solved.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
BFS: fewest edges, grids, many sources	Ch. 15	1 day ago	☐ Again ☐ Shaky ☐ Solid
DFS without recursion, components	Ch. 15	1 day ago	☐ Again ☐ Shaky ☐ Solid
Topological order (Kahn) and DAG DP	Ch. 15	1 day ago	☐ Again ☐ Shaky ☐ Solid
0-1 BFS	Ch. 15	1 day ago	☐ Again ☐ Shaky ☐ Solid
Longest increasing subsequence	Ch. 13	3 days ago	☐ Again ☐ Shaky ☐ Solid
Brute force first, then stress test	Ch. 9	7 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Prefix sums + HashMap	Ch. 2	14 days ago	☐ Again ☐ Shaky ☐ Solid
Difference array (many range adds)	Ch. 2	14 days ago	☐ Again ☐ Shaky ☐ Solid

Mock exam: Mock 2 · number theory and DP added

Q1 ≈ 1300, Q2 ≈ 1600, Q3 ≈ 1900. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- ☐ **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- ☐ **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- ☐ **By minute 150** every problem has a submission, even if it is only a brute force.
- ☐ **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

Planning grid (first 15 minutes)

Q	Limits	Idea	Complexity	Brute-force fallback	Order
Q1					
Q2					
Q3					

Score sheet

Q	Problem	Rating	Weight	Start	End	% tests	Marks
Q1	**CF 1367C** Social Distance codeforces.com/problemset/problem/1367/C	1300	20				
Q2	**CF 1373D** Maximum Sum on Even Positions codeforces.com/problemset/problem/1373/D	1600	30				
Q3	**CF 1475G** Strange Beauty codeforces.com/problemset/problem/1475/G	1900	50				
	Weighted score = Σ weight × % tests passed						

SP checkpoint. Q1 + Q2 full, weighted score ≥ 50/100.

Post-mortem (about 60 minutes after the mock)

Q	What went wrong	Cause	Rule that would have prevented it
Q1		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q2		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q3		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	

Re-solve from three days ago

Pick one problem from Day 13 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- [ ] 13.P1 LC 300 · Longest Increasing Subsequence
- [ ] 13.P2 LC 1964 · Find the Longest Valid Obstacle Course at Each Position
- [ ] 13.P3 LC 1671 · Minimum Number of Removals to Make Mountain Array

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 17, Graphs II: Dijkstra, DSU, MST.

DAY 17 OF 29 · MONDAY 23 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

17

Graphs II: Dijkstra, DSU, MST

Weighted shortest paths, expanded state graphs, union-find and Kruskal.

Read first · Theory Chapter 17
Patterns · Dijkstra, Union-find (DSU) and Kruskal

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 17; type each new listing once and run it	40 min
☐	17.P1 LC 743 · Network Delay Time	40 min
☐	17.P2 LC 1631 · Path With Minimum Effort	40 min
☐	17.P3 LC 1584 · Min Cost to Connect All Points	40 min
☐	17.P4 LC 1976 · Number of Ways to Arrive at Destination	40 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 17.P5, 17.P6, 17.P7*	*60 min*
	Core total (estimate)	**3.8 h**

NOTES FOR THE DAY

- Dijkstra with `PriorityQueue<long[]>`; skip stale entries (`if (d != dist[u]) continue`).
- Expanded state: a node becomes (vertex, stops used) or (vertex, coupon used).
- Counting shortest paths: when you find an equal distance, add the ways.
- Kruskal = sort edges + DSU.

Why it matters for Infosys. "Connect cities at minimum cost" (MST) and Dijkstra are both repeat topics.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Interval DP and game DP	Ch. 14	3 days ago	☐ Again ☐ Shaky ☐ Solid
gcd, fast power, inverse, nCr	Ch. 10	7 days ago	☐ Again ☐ Shaky ☐ Solid
Sieve, prime factors, square-free kernel	Ch. 10	7 days ago	☐ Again ☐ Shaky ☐ Solid
Two pointers on a sorted array	Ch. 3	14 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Sliding window (variable size)	Ch. 3	14 days ago	☐ Again ☐ Shaky ☐ Solid
Window maximum (monotonic deque)	Ch. 3	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

17.P1 LC 743 · Network Delay Time MEDIUM CORE — **40 min**

`leetcode.com/problems/network-delay-time` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P1*

17.P2 LC 1631 · Path With Minimum Effort MEDIUM CORE — **40 min**

`leetcode.com/problems/path-with-minimum-effort` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P2*

17.P3 LC 1584 · Min Cost to Connect All Points MEDIUM CORE — **40 min**

`leetcode.com/problems/min-cost-to-connect-all-points` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P3*

17.P4 LC 1976 · Number of Ways to Arrive at Destination MEDIUM CORE — 40 min

leetcode.com/problems/number-of-ways-to-arrive-at-destination · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P4*

17.P5 LC 787 · Cheapest Flights Within K Stops MEDIUM BONUS — 40 min

leetcode.com/problems/cheapest-flights-within-k-stops · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P5*

17.P6 LC 2290 · Minimum Obstacle Removal to Reach Corner HARD BONUS — 60 min

leetcode.com/problems/minimum-obstacle-removal-to-reach-corner · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P6*

17.P7 LC 2421 · Number of Good Paths HARD BONUS — 60 min

leetcode.com/problems/number-of-good-paths · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P7*

Re-solve from three days ago

Pick one problem from Day 14 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 14.P1 LC 368 · Largest Divisible Subset ☐ 14.P2 LC 486 · Predict the Winner
☐ 14.P3 LC 1547 · Minimum Cost to Cut a Stick ☐ 14.P4 LC 1690 · Stone Game VII

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 18, Trees I: recursion, parent arrays, LCA.

DAY 18 OF 29 · TUESDAY 24 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

18

Trees I: recursion, parent arrays, LCA

LeetCode TreeNode recursion, general trees without recursion, and binary lifting.

Read first · Theory Chapter 18
Patterns · Binary tree recursion (LeetCode TreeNode), General trees from a parent array (no recursion), LCA with binary lifting

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 18; type each new listing once and run it	40 min
☐	18.P1 LC 543 · Diameter of Binary Tree	20 min
☐	18.P2 LC 236 · Lowest Common Ancestor of a Binary Tree	40 min
☐	18.P3 LC 124 · Binary Tree Maximum Path Sum	60 min
☐	18.P4 LC 2246 · Longest Path With Different Adjacent Characters	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 18.P5, 18.P6, 18.P7, 18.P8*	*60 min*
	Core total (estimate)	**4.1 h**

NOTES FOR THE DAY

- Binary trees: decide what one node returns to its parent; paths that bend go into a "best" holder.
- General trees with 10^5 nodes: build the BFS order and walk it backwards. No recursion, no stack overflow.
- Binary lifting: `up[k][v] = up[k-1][up[k-1][v]]` ; dist(a, b) = depth[a] + depth[b] − 2·depth[lca].

Why it matters for Infosys. Both trees in the sample paper arrive as parent arrays with n = 10^5. A recursive DFS on a 10^5-node chain crashes in Java.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Dijkstra	Ch. 17	1 day ago	☐ Again ☐ Shaky ☐ Solid
Union-find (DSU) and Kruskal	Ch. 17	1 day ago	☐ Again ☐ Shaky ☐ Solid
BFS: fewest edges, grids, many sources	Ch. 15	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
DFS without recursion, components	Ch. 15	3 days ago	☐ Again ☐ Shaky ☐ Solid
Topological order (Kahn) and DAG DP	Ch. 15	3 days ago	☐ Again ☐ Shaky ☐ Solid
0-1 BFS	Ch. 15	3 days ago	☐ Again ☐ Shaky ☐ Solid
DP recipe: state, transition, base, answer	Ch. 11	7 days ago	☐ Again ☐ Shaky ☐ Solid
Knapsack family	Ch. 11	7 days ago	☐ Again ☐ Shaky ☐ Solid
Intervals: sort, then sweep	Ch. 4	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 4 bonus.

18.P1 LC 543 · Diameter of Binary Tree EASY CORE — 20 min

leetcode.com/problems/diameter-of-binary-tree · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 18.P1*

18.P2 LC 236 · Lowest Common Ancestor of a Binary Tree MEDIUM CORE — 40 min

leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 18.P2*

18.P3 LC 124 · Binary Tree Maximum Path Sum HARD CORE — 60 min

leetcode.com/problems/binary-tree-maximum-path-sum · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 18.P3*

18.P4 LC 2246 · Longest Path With Different Adjacent Characters HARD CORE

60 min

leetcode.com/problems/longest-path-with-different-adjacent-characters · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 18.P4

18.P5 LC 102 · Binary Tree Level Order Traversal MEDIUM BONUS

40 min

leetcode.com/problems/binary-tree-level-order-traversal · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 18.P5

18.P6 LC 1483 · Kth Ancestor of a Tree Node HARD BONUS

60 min

leetcode.com/problems/kth-ancestor-of-a-tree-node · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 18.P6

18.P7 LC 2477 · Minimum Fuel Cost to Report to the Capital MEDIUM BONUS

40 min

leetcode.com/problems/minimum-fuel-cost-to-report-to-the-capital · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 18.P7

18.P8 **LC 2846 · Minimum Edge Weight Equilibrium Queries in a Tree** HARD BONUS 60 min

leetcode.com/problems/minimum-edge-weight-equilibrium-queries-in-a-tree · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 18.P8*

Re-solve from three days ago

Pick one problem from Day 15 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 15.P1 LC 200 · Number of Islands ☐ 15.P2 LC 994 · Rotting Oranges ☐ 15.P3 LC 207 · Course Schedule
☐ 15.P4 LC 1091 · Shortest Path in Binary Matrix

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 19, Trees II: tree DP, rerooting, small-to-large.

DAY 19 OF 29 · WEDNESDAY 25 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

19

Trees II: tree DP, rerooting, small-to-large

dp[v][state] from the children, answers for every root in O(n), and merging the smaller set into the larger.

Read first · Theory Chapter 19
Patterns · Tree DP (dp[v][state]), Rerooting (an answer for every root), Small-to-large merging

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 19; type each new listing once and run it	40 min
☐	19.P1 LC 337 · House Robber III	40 min
☐	19.P2 LC 1519 · Number of Nodes in the Sub-Tree With the Same Label	40 min
☐	19.P3 LC 834 · Sum of Distances in Tree	60 min
☐	19.P4 LC 3068 · Find the Maximum Sum of Node Values	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 19.P5, 19.P6, 19.P7*	*60 min*
	Core total (estimate)	**4.4 h**

NOTES FOR THE DAY

- Tree DP: a few states per node (taken / not taken), combined from the children.
- Rerooting: pass 1 computes subtree answers; pass 2 moves the root to each child with a formula.
- Small-to-large: keep the bigger child set and pour the smaller sets into it. This is exactly the sample Easy problem.

Why it matters for Infosys. The sample Easy is small-to-large, and the sample Complex changes both ends of an edge (like LC 3068). This is the most valuable day for SP.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Binary tree recursion (LeetCode TreeNode)	Ch. 18	1 day ago	☐ Again ☐ Shaky ☐ Solid
General trees from a parent array (no recursion)	Ch. 18	1 day ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
LCA with binary lifting	Ch. 18	1 day ago	☐ Again ☐ Shaky ☐ Solid
Two-string DP (LCS, edit distance)	Ch. 12	7 days ago	☐ Again ☐ Shaky ☐ Solid
Greedy with a heap	Ch. 5	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

19.P1 LC 337 · House Robber III MEDIUM CORE — 40 min

leetcode.com/problems/house-robber-iii · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P1*

19.P2 LC 1519 · Number of Nodes in the Sub-Tree With the Same Label MEDIUM CORE — 40 min

leetcode.com/problems/number-of-nodes-in-the-sub-tree-with-the-same-label · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P2*

19.P3 LC 834 · Sum of Distances in Tree HARD CORE — 60 min

leetcode.com/problems/sum-of-distances-in-tree · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P3*

19.P4 LC 3068 · Find the Maximum Sum of Node Values HARD CORE — 60 min

leetcode.com/problems/find-the-maximum-sum-of-node-values · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P4*

19.P5 LC 2858 · Minimum Edge Reversals So Every Node Is Reachable HARD BONUS — 60 min

leetcode.com/problems/minimum-edge-reversals-so-every-node-is-reachable · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P5*

19.P6 LC 2003 · Smallest Missing Genetic Value in Each Subtree HARD BONUS — 60 min

leetcode.com/problems/smallest-missing-genetic-value-in-each-subtree · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P6*

19.P7 CF 600E · Lomsat gelral CF 2300 BONUS — 75 min

codeforces.com/problemset/problem/600/E · Codeforces

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P7*

Re-solve from three days ago

Pick one problem from Day 16 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 16.P1 CF 1367C · Social Distance ☐ 16.P2 CF 1373D · Maximum Sum on Even Positions
☐ 16.P3 CF 1475G · Strange Beauty

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 20, Fenwick trees and segment trees.

DAY 20 OF 29 · THURSDAY 26 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

20

Fenwick trees and segment trees

Point updates with range queries for sum, min and gcd, coordinate compression, and Euler tours.

Read first · Theory Chapter 20
Patterns · Fenwick tree (BIT), Segment tree (iterative), Euler tour (subtree = range)

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 20; type each new listing once and run it	40 min
☐	20.P1 LC 307 · Range Sum Query - Mutable	40 min
☐	20.P2 LC 315 · Count of Smaller Numbers After Self	60 min
☐	20.P3 LC 2407 · Longest Increasing Subsequence II	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 20.P4, 20.P5, 20.P6, 20.P7*	*60 min*
	Core total (estimate)	**4.0 h**

NOTES FOR THE DAY

- Fenwick: point add plus prefix sum. "Count smaller to the right": scan right to left over compressed values.
- The iterative segment tree handles any associative operation: change `op` and `ID`.
- An Euler tour turns every subtree into one range, so subtree questions become range questions.

Why it matters for Infosys. The sample Medium is a gcd segment tree plus one counter. CF 914D (bonus) is the closest public problem to it.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Tree DP (dp[v][state])	Ch. 19	1 day ago	☐ Again ☐ Shaky ☐ Solid
Rerooting (an answer for every root)	Ch. 19	1 day ago	☐ Again ☐ Shaky ☐ Solid
Small-to-large merging	Ch. 19	1 day ago	☐ Again ☐ Shaky ☐ Solid
Dijkstra	Ch. 17	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Union-find (DSU) and Kruskal	Ch. 17	3 days ago	☐ Again ☐ Shaky ☐ Solid
Longest increasing subsequence	Ch. 13	7 days ago	☐ Again ☐ Shaky ☐ Solid
Binary search: lower and upper bound	Ch. 6	14 days ago	☐ Again ☐ Shaky ☐ Solid
Binary search on the answer	Ch. 6	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 3 core problems, 4 bonus.

20.P1 LC 307 · Range Sum Query - Mutable MEDIUM CORE — 40 min

leetcode.com/problems/range-sum-query-mutable · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P1*

20.P2 LC 315 · Count of Smaller Numbers After Self HARD CORE — 60 min

leetcode.com/problems/count-of-smaller-numbers-after-self · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P2*

20.P3 LC 2407 · Longest Increasing Subsequence II HARD CORE — 60 min

leetcode.com/problems/longest-increasing-subsequence-ii · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P3*

20.P4 **CF 914D · Bash and a Tough Math Puzzle** CF 1900 BONUS — 60 min

codeforces.com/problemset/problem/914/D · Codeforces

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P4*

20.P5 **LC 2286 · Booking Concert Tickets in Groups** HARD BONUS — 60 min

leetcode.com/problems/booking-concert-tickets-in-groups · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P5*

20.P6 **LC 3165 · Maximum Sum of Subsequence With Non-adjacent Elements** HARD BONUS — 60 min

leetcode.com/problems/maximum-sum-of-subsequence-with-non-adjacent-elements · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P6*

20.P7 **LC 2322 · Minimum Score After Removals on a Tree** HARD BONUS — 60 min

leetcode.com/problems/minimum-score-after-removals-on-a-tree · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P7*

Re-solve from three days ago

Pick one problem from Day 17 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 17.P1 LC 743 · Network Delay Time ☐ 17.P2 LC 1631 · Path With Minimum Effort
☐ 17.P3 LC 1584 · Min Cost to Connect All Points ☐ 17.P4 LC 1976 · Number of Ways to Arrive at Destination

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 21, Lazy segment trees and the K-parts DP.

DAY 21 OF 29 · FRIDAY 27 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

21

Lazy segment trees and the K-parts DP

Range updates, then the split-into-K-parts DP that Infosys reuses for its hardest problems.

Read first · Theory Chapter 21
Patterns · Lazy segment tree (range add, range max), Split into K parts (dp[k][i])

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 21; type each new listing once and run it	40 min
☐	21.P1 LC 1335 · Minimum Difficulty of a Job Schedule	60 min
☐	21.P2 LC 813 · Largest Sum of Averages	40 min
☐	21.P3 LC 2916 · Subarrays Distinct Element Sum of Squares II	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 21.P4, 21.P5, 21.P6*	*60 min*
	Core total (estimate)	**4.0 h**

NOTES FOR THE DAY

- Lazy propagation: store a pending add on a node and push it down before visiting its children.
- K parts: `dp[k][i] = best over j of dp[k-1][j] + cost(j, i-1)`. $O(K \cdot n^2)$ gets the small tests.
- When moving i changes cost by a range add over j (distinct counts do), keep dp + cost in a lazy segment tree.

Why it matters for Infosys. The Infosys "gift boxes" problem is CF 833B and "k segments, minimum cost" is CF 1527E, with the same limits. Both are today's bonus problems.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Euler tour (subtree = range)	Ch. 20	1 day ago	☐ Again ☐ Shaky ☐ Solid
Fenwick tree (BIT)	Ch. 20	1 day ago	☐ Again ☐ Shaky ☐ Solid
Segment tree (iterative)	Ch. 20	1 day ago	☐ Again ☐ Shaky ☐ Solid
Binary tree recursion (LeetCode TreeNode)	Ch. 18	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
General trees from a parent array (no recursion)	Ch. 18	3 days ago	☐ Again ☐ Shaky ☐ Solid
LCA with binary lifting	Ch. 18	3 days ago	☐ Again ☐ Shaky ☐ Solid
Interval DP and game DP	Ch. 14	7 days ago	☐ Again ☐ Shaky ☐ Solid
Monotonic stack	Ch. 7	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 3 core problems, 3 bonus.

21.P1 LC 1335 · Minimum Difficulty of a Job Schedule HARD CORE — **60 min**

`leetcode.com/problems/minimum-difficulty-of-a-job-schedule` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 21.P1*

21.P2 LC 813 · Largest Sum of Averages MEDIUM CORE — **40 min**

`leetcode.com/problems/largest-sum-of-averages` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 21.P2*

21.P3 LC 2916 · Subarrays Distinct Element Sum of Squares II HARD CORE — **60 min**

`leetcode.com/problems/subarrays-distinct-element-sum-of-squares-ii` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 21.P3*

21.P4 **CF 833B · The Bakery** CF 2200 BONUS — **75 min**

codeforces.com/problemset/problem/833/B · Codeforces

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 21.P4*

21.P5 **LC 732 · My Calendar III** HARD BONUS — **60 min**

leetcode.com/problems/my-calendar-iii · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 21.P5*

21.P6 **CF 1527E · Partition Game** CF 2500 BONUS — **75 min**

codeforces.com/problemset/problem/1527/E · Codeforces

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 21.P6*

Re-solve from three days ago

Pick one problem from Day 18 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 18.P1 LC 543 · Diameter of Binary Tree ☐ 18.P2 LC 236 · Lowest Common Ancestor of a Binary Tree
☐ 18.P3 LC 124 · Binary Tree Maximum Path Sum ☐ 18.P4 LC 2246 · Longest Path With Different Adjacent Characters

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 22, Backtracking, bitmask DP, digit DP.

DAY 22 OF 29 · SATURDAY 28 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

22

Backtracking, bitmask DP, digit DP

Brute force for partial marks, dp[mask] for n ≤ 20, and counting numbers digit by digit.

Read first · Theory Chapter 22
Patterns · Backtracking and subset enumeration, Bitmask DP, Digit DP

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 22; type each new listing once and run it	40 min
☐	22.P1 LC 78 · Subsets	40 min
☐	22.P2 LC 1986 · Minimum Number of Work Sessions to Finish the Tasks	40 min
☐	22.P3 LC 1879 · Minimum XOR Sum of Two Arrays	60 min
☐	22.P4 LC 902 · Numbers At Most N Given Digit Set	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 22.P5, 22.P6, 22.P7*	*60 min*
	Core total (estimate)	**4.4 h**

NOTES FOR THE DAY

- Choose, recurse, un-choose. A bitmask loop enumerates every subset for n ≤ 20.
- Bitmask DP: `dp[mask]` or `dp[mask][last]`, masks in increasing order; $2^{20} \cdot 20 \approx 2 \cdot 10^7$.
- Digit DP: state (position, tight, extra); answer for [L, R] = f(R) − f(L − 1).

Why it matters for Infosys. Reported recent hard problems include "tree + DP + backtracking + bitwise" and a bitmask-DP graph problem.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Lazy segment tree (range add, range max)	Ch. 21	1 day ago	☐ Again ☐ Shaky ☐ Solid
Split into K parts (dp[k][i])	Ch. 21	1 day ago	☐ Again ☐ Shaky ☐ Solid
Tree DP (dp[v][state])	Ch. 19	3 days ago	☐ Again ☐ Shaky ☐ Solid
Rerooting (an answer for every root)	Ch. 19	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Small-to-large merging	Ch. 19	3 days ago	☐ Again ☐ Shaky ☐ Solid
BFS: fewest edges, grids, many sources	Ch. 15	7 days ago	☐ Again ☐ Shaky ☐ Solid
DFS without recursion, components	Ch. 15	7 days ago	☐ Again ☐ Shaky ☐ Solid
Topological order (Kahn) and DAG DP	Ch. 15	7 days ago	☐ Again ☐ Shaky ☐ Solid
0-1 BFS	Ch. 15	7 days ago	☐ Again ☐ Shaky ☐ Solid
Bit tricks and XOR	Ch. 8	14 days ago	☐ Again ☐ Shaky ☐ Solid
Binary trie for XOR	Ch. 8	14 days ago	☐ Again ☐ Shaky ☐ Solid
XOR basis (maximum subset XOR)	Ch. 8	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

22.P1 LC 78 · Subsets MEDIUM CORE — **40 min**

`leetcode.com/problems/subsets` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P1*

22.P2 LC 1986 · Minimum Number of Work Sessions to Finish the Tasks MEDIUM CORE — **40 min**

`leetcode.com/problems/minimum-number-of-work-sessions-to-finish-the-tasks` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P2*

22.P3 LC 1879 · Minimum XOR Sum of Two Arrays HARD CORE — 60 min

leetcode.com/problems/minimum-xor-sum-of-two-arrays · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 22.P3

22.P4 LC 902 · Numbers At Most N Given Digit Set HARD CORE — 60 min

leetcode.com/problems/numbers-at-most-n-given-digit-set · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 22.P4

22.P5 LC 51 · N-Queens HARD BONUS — 60 min

leetcode.com/problems/n-queens · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 22.P5

22.P6 LC 847 · Shortest Path Visiting All Nodes HARD BONUS — 60 min

leetcode.com/problems/shortest-path-visiting-all-nodes · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

Appendix A, hint 22.P6

22.P7 **LC 2827 · Number of Beautiful Integers in the Range** HARD BONUS — 60 min

leetcode.com/problems/number-of-beautiful-integers-in-the-range · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P7*

Re-solve from three days ago

Pick one problem from Day 19 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 19.P1 LC 337 · House Robber III ☐ 19.P2 LC 1519 · Number of Nodes in the Sub-Tree With the Same Label
☐ 19.P3 LC 834 · Sum of Distances in Tree ☐ 19.P4 LC 3068 · Find the Maximum Sum of Node Values

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 23, Mock 3 (4 problems) + review.

DAY 23 OF 29 · SUNDAY 29 NOVEMBER 2026 · MOCK DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES 23

Mock 3 (4 problems) + review

Full SP-level mock with a 4th problem, as in recent in-person rounds. This date is also the fallback exam slot.

Read first · Theory Chapter 23
Patterns · —

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	23.P1 CF 1426D · Non-zero Segments	45 min
☐	23.P2 CF 1689C · Infected Tree	45 min
☐	23.P3 CF 1187E · Tree Painting	75 min
☐	23.P4 CF 1913D · Array Collapse	75 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
	Core total (estimate)	**4.0 h**

NOTES FOR THE DAY

- Four problems in 3 hours. Aim for full marks on 2–3 and a brute force on the rest.
- After the mock: scores, editorials, re-code, error log.

Why it matters for Infosys. The SP L3 report was 3 full solutions plus 6/12 tests on the 4th. Practise collecting partial marks on Q4 here.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Backtracking and subset enumeration	Ch. 22	1 day ago	☐ Again ☐ Shaky ☐ Solid
Bitmask DP	Ch. 22	1 day ago	☐ Again ☐ Shaky ☐ Solid
Digit DP	Ch. 22	1 day ago	☐ Again ☐ Shaky ☐ Solid
Euler tour (subtree = range)	Ch. 20	3 days ago	☐ Again ☐ Shaky ☐ Solid
Fenwick tree (BIT)	Ch. 20	3 days ago	☐ Again ☐ Shaky ☐ Solid
Segment tree (iterative)	Ch. 20	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Brute force first, then stress test	Ch. 9	14 days ago	☐ Again ☐ Shaky ☐ Solid

Mock exam: Mock 3 · four problems

Q1 ≈ 1500, Q2 ≈ 1600, Q3 ≈ 2100, Q4 ≈ 2100. Collect partial marks on Q4. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- ☐ **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- ☐ **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- ☐ **By minute 150** every problem has a submission, even if it is only a brute force.
- ☐ **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

Planning grid (first 15 minutes)

Q	Limits	Idea	Complexity	Brute-force fallback	Order
Q1					
Q2					
Q3					
Q4					

Score sheet

Q	Problem	Rating	Weight	Start	End	% tests	Marks
Q1	**CF 1426D** Non-zero Segments `codeforces.com/problemset/problem/1426/D`	1500	20				
Q2	**CF 1689C** Infected Tree `codeforces.com/problemset/problem/1689/C`	1600	30				
Q3	**CF 1187E** Tree Painting `codeforces.com/problemset/problem/1187/E`	2100	50				
Q4	**CF 1913D** Array Collapse `codeforces.com/problemset/problem/1913/D`	2100	partial				
	Weighted score = Σ weight × % tests passed						

SP checkpoint. Weighted score ≥ 60 and a brute force submitted on every problem.

Post-mortem (about 60 minutes after the mock)

Q	What went wrong	Cause	Rule that would have prevented it
Q1		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q2		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q3		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q4		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	

Re-solve from three days ago

Pick one problem from Day 20 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- [ ] 20.P1 LC 307 · Range Sum Query - Mutable
- [ ] 20.P2 LC 315 · Count of Smaller Numbers After Self
- [ ] 20.P3 LC 2407 · Longest Increasing Subsequence II

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 24, Infosys repeat problems I.

DAY 24 OF 29 · MONDAY 30 NOVEMBER 2026 · LEARNING DAY · PHASE 4 · EXAM SIMULATION

24

Infosys repeat problems I

The most reported Infosys problems, written as full programs with one value per line and checked with the stress tester.

Read first · Theory Chapter 24
Patterns · —

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 24; type each new listing once and run it	40 min
☐	24.P1 RPG: maximum monsters defeated	30 min
☐	24.P2 Andy's vacation (cancel ≤ K obligations)	30 min
☐	24.P3 Gym: fewest exercises to drain energy	20 min
☐	24.P4 GfG · Wine Buying and Selling	30 min
☐	24.P5 Arrays where each element divides the next (mod 10000)	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 24.P6, 24.P7, 24.P8, 24.P9, 24.P10*	*60 min*
	Core total (estimate)	**3.9 h**

NOTES FOR THE DAY

- For each problem without a judge: write the brute force, then the fast solution, then stress-test until thousands of random cases match. OneCompiler runs both.
- Read the input exactly as the card says: one value per line, in the order of the samples.
- At most 40 minutes per problem before you open the folded approach.

Why it matters for Infosys. Several of these were reported in 4–6 different years.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Lazy segment tree (range add, range max)	Ch. 21	3 days ago	☐ Again ☐ Shaky ☐ Solid
Split into K parts (dp[k][i])	Ch. 21	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Dijkstra	Ch. 17	7 days ago	☐ Again ☐ Shaky ☐ Solid
Union-find (DSU) and Kruskal	Ch. 17	7 days ago	☐ Again ☐ Shaky ☐ Solid
gcd, fast power, inverse, nCr	Ch. 10	14 days ago	☐ Again ☐ Shaky ☐ Solid
Sieve, prime factors, square-free kernel	Ch. 10	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 5 core problems, 5 bonus.

24.P1 RPG: maximum monsters defeated SELF-JUDGE CORE — **30 min**

`No online judge · solve in OneCompiler · Self-judge`

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P1*

24.P2 Andy's vacation (cancel ≤ K obligations) SELF-JUDGE CORE — **30 min**

`No online judge · solve in OneCompiler · Self-judge`

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P2*

24.P3 Gym: fewest exercises to drain energy SELF-JUDGE CORE — **20 min**

`No online judge · solve in OneCompiler · Self-judge`

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P3*

24.P4 GfG · Wine Buying and Selling GFG CORE — 30 min

geeksforgeeks.org/problems/wine-buying-and-selling/1 · GeeksforGeeks

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P4*

24.P5 Arrays where each element divides the next (mod 10000) SELF-JUDGE CORE — 60 min

No online judge · solve in OneCompiler · Self-judge

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P5*

24.P6 LIS with (x & y)·2 < (x | y) SELF-JUDGE BONUS — 45 min

No online judge · solve in OneCompiler · Self-judge

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P6*

24.P7 Smallest base with identical digits SELF-JUDGE BONUS — 45 min

No online judge · solve in OneCompiler · Self-judge

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P7*

24.P8 **Choose X ≤ K maximising Σ X xor A[i]** SELF-JUDGE BONUS — 45 min

No online judge · solve in OneCompiler · Self-judge

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P8*

24.P9 **CF 1338A · Powered Addition** CF 1500 BONUS — 45 min

codeforces.com/problemset/problem/1338/A · Codeforces

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P9*

24.P10 **CF 1514C · Product 1 Modulo N** CF 1600 BONUS — 45 min

codeforces.com/problemset/problem/1514/C · Codeforces

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P10*

Re-solve from three days ago

Pick one problem from Day 21 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 21.P1 LC 1335 · Minimum Difficulty of a Job Schedule ☐ 21.P2 LC 813 · Largest Sum of Averages

☐ 21.P3 LC 2916 · Subarrays Distinct Element Sum of Squares II

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 25, Mock 4: the official Infosys sample paper.

DAY 25 OF 29 · TUESDAY 1 DECEMBER 2026 · MOCK DAY · PHASE 4 · EXAM SIMULATION

25

Mock 4: the official Infosys sample paper

Infosys's own paper under exam rules. Grade yourself with the samples and your brute force, then open the walkthroughs.

Read first · Theory Chapter 25
Patterns · —

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	25.P1 Sample paper · Easy (tree beauty)	45 min
☐	25.P2 Sample paper · Medium (gcd under updates)	45 min
☐	25.P3 Sample paper · Hard (LNDS with XOR ≥ M)	60 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 25.P4*	*30 min*
	Core total (estimate)	**4.0 h**

NOTES FOR THE DAY

- Download the sample PDF and copy the sample inputs.
- Write each solution in OneCompiler; run it with the samples in the STDIN box.
- Solve Easy, Medium and Hard in 3 hours. Attempt Complex only for partial marks.
- Grade: samples, plus a stress test against your own brute force. Only then read the walkthroughs in Theory Chapter 25.

Why it matters for Infosys. This is the closest thing to the real paper you will have.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Backtracking and subset enumeration	Ch. 22	3 days ago	☐ Again ☐ Shaky ☐ Solid
Bitmask DP	Ch. 22	3 days ago	☐ Again ☐ Shaky ☐ Solid
Digit DP	Ch. 22	3 days ago	☐ Again ☐ Shaky ☐ Solid

Pattern	Taught	Interval	Rating
Binary tree recursion (LeetCode TreeNode)	Ch. 18	7 days ago	☐ Again ☐ Shaky ☐ Solid
General trees from a parent array (no recursion)	Ch. 18	7 days ago	☐ Again ☐ Shaky ☐ Solid
LCA with binary lifting	Ch. 18	7 days ago	☐ Again ☐ Shaky ☐ Solid
DP recipe: state, transition, base, answer	Ch. 11	14 days ago	☐ Again ☐ Shaky ☐ Solid
Knapsack family	Ch. 11	14 days ago	☐ Again ☐ Shaky ☐ Solid

Mock exam: Mock 4 · official Infosys sample paper

Infosys's own paper. Run it in OneCompiler with the PDF's samples, grade with your stress tester, then open the walk-throughs. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- ☐ **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- ☐ **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- ☐ **By minute 150** every problem has a submission, even if it is only a brute force.
- ☐ **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

Planning grid (first 15 minutes)

Q	Limits	Idea	Complexity	Brute-force fallback	Order
Q1					
Q2					
Q3					
Q4					

Score sheet

Q	Problem	Rating	Weight	Start	End	% tests	Marks
Q1	**Infosys** Sample paper · Easy (tree beauty) `rec-test.infosys.com/iwa-web/public/instructions/Sample_Test_SP_DSE.pdf`	—	20				
Q2	**Infosys** Sample paper · Medium (gcd under updates) `rec-test.infosys.com/iwa-web/public/instructions/Sample_Test_SP_DSE.pdf`	—	30				
	Weighted score = Σ weight × % tests passed						

Q	Problem	Rating	Weight	Start	End	% tests	Marks
Q3	**Infosys** Sample paper · Hard (LNDS with XOR ≥ M) `rec-test.infosys.com/iwa-web/public/instructions/Sample_Test_SP_DSE.pdf`	—	50				
Q4	**Infosys** Sample paper · Complex (partial marks only) `rec-test.infosys.com/iwa-web/public/instructions/Sample_Test_SP_DSE.pdf`	—	partial				
	Weighted score = Σ weight × % tests passed						

SP checkpoint. Compare with the walkthroughs in Theory Chapter 25 only after grading yourself.

Post-mortem (about 60 minutes after the mock)

Q	What went wrong	Cause	Rule that would have prevented it
Q1		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q2		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q3		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q4		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	

Re-solve from three days ago

Pick one problem from Day 22 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- ☐ 22.P1 LC 78 · Subsets
- ☐ 22.P2 LC 1986 · Minimum Number of Work Sessions to Finish the Tasks
- ☐ 22.P3 LC 1879 · Minimum XOR Sum of Two Arrays
- ☐ 22.P4 LC 902 · Numbers At Most N Given Digit Set

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 26, Repair day + the hard XOR repeat.

DAY 26 OF 29 · WEDNESDAY 2 DECEMBER 2026 · LEARNING DAY · PHASE 4 · EXAM SIMULATION

26

Repair day + the hard XOR repeat

Fix what the mocks exposed, then the hardest XOR repeat.

Read first · Theory Chapter 26
Patterns · —

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 26; type each new listing once and run it	40 min
☐	26.P1 Max XOR choosing at most N/2 elements	60 min
☐	26.P2 Pairs with XOR < K and a prime sum	45 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
☐	*Bonus hour (optional, only if the core is done and you feel sharp): 26.P3*	*45 min*
	Core total (estimate)	**3.8 h**

NOTES FOR THE DAY

- First 90 minutes: re-solve every error-log entry dated 3+ days ago, without notes.
- Then the problems below.
- This is a lighter day on purpose. Stop when the core is done.

Why it matters for Infosys. This late, most of the score gain comes from removing your own repeated mistakes, not from new topics.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Tree DP (dp[v][state])	Ch. 19	7 days ago	☐ Again ☐ Shaky ☐ Solid
Rerooting (an answer for every root)	Ch. 19	7 days ago	☐ Again ☐ Shaky ☐ Solid
Small-to-large merging	Ch. 19	7 days ago	☐ Again ☐ Shaky ☐ Solid
Two-string DP (LCS, edit distance)	Ch. 12	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 2 core problems, 1 bonus.

26.P1 Max XOR choosing at most N/2 elements SELF-JUDGE CORE — 60 min

No online judge · solve in OneCompiler · Self-judge

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 26.P1*

26.P2 Pairs with XOR < K and a prime sum SELF-JUDGE CORE — 45 min

No online judge · solve in OneCompiler · Self-judge

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 26.P2*

26.P3 Sample Complex: brute force for N ≤ 15 SELF-JUDGE BONUS — 45 min

No online judge · solve in OneCompiler · Self-judge

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 26.P3*

Re-solve from three days ago

Pick one problem from Day 23 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 23.P1 CF 1426D · Non-zero Segments ☐ 23.P2 CF 1689C · Infected Tree ☐ 23.P3 CF 1187E · Tree Painting
☐ 23.P4 CF 1913D · Array Collapse

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 27, Mock 5: dress rehearsal.

DAY 27 OF 29 · THURSDAY 3 DECEMBER 2026 · MOCK DAY · PHASE 4 · EXAM SIMULATION

27

Mock 5: dress rehearsal

Start at your real reporting time, 4 problems, paper notes for the first 15 minutes, exact exam rules.

Read first · Theory Chapter 27
Patterns · —

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	27.P1 CF 1385D · a-Good String	45 min
☐	27.P2 CF 1446B · Catching Cheaters	60 min
☐	27.P3 CF 1324F · Maximum White Subtree	60 min
☐	27.P4 CF 1433F · Zero Remainder Sum	75 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
	Core total (estimate)	**4.0 h**

NOTES FOR THE DAY

- Start at the same clock time as your reporting time on 6 Dec.
- No music, no snacks at the desk, phone out of the room.
- Afternoon: review only. No new problems after this mock.

Why it matters for Infosys. This is the last full rehearsal. After today, protect your confidence and your sleep.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Euler tour (subtree = range)	Ch. 20	7 days ago	☐ Again ☐ Shaky ☐ Solid
Fenwick tree (BIT)	Ch. 20	7 days ago	☐ Again ☐ Shaky ☐ Solid
Segment tree (iterative)	Ch. 20	7 days ago	☐ Again ☐ Shaky ☐ Solid
Longest increasing subsequence	Ch. 13	14 days ago	☐ Again ☐ Shaky ☐ Solid

Mock exam: Mock 5 · dress rehearsal

Q1 ≈ 1500, Q2 ≈ 1800, Q3 ≈ 1800, Q4 ≈ 2100. Start at your real reporting time. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- [ ] **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- [ ] **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- [ ] **By minute 150** every problem has a submission, even if it is only a brute force.
- [ ] **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

Planning grid (first 15 minutes)

Q	Limits	Idea	Complexity	Brute-force fallback	Order
Q1					
Q2					
Q3					
Q4					

Score sheet

Q	Problem	Rating	Weight	Start	End	% tests	Marks
Q1	**CF 1385D** a-Good String codeforces.com/problemset/problem/1385/D	1500	20				
Q2	**CF 1446B** Catching Cheaters codeforces.com/problemset/problem/1446/B	1800	30				
Q3	**CF 1324F** Maximum White Subtree codeforces.com/problemset/problem/1324/F	1800	50				
Q4	**CF 1433F** Zero Remainder Sum codeforces.com/problemset/problem/1433/F	2100	partial				
	Weighted score = Σ weight × % tests passed						

SP checkpoint. Weighted score ≥ 65.

Post-mortem (about 60 minutes after the mock)

Q	What went wrong	Cause	Rule that would have prevented it
Q1		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q2		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q3		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	
Q4		misread · idea · overflow · off-by-one · TLE · input · edge · stack · time	

Re-solve from three days ago

Pick one problem from Day 24 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- [ ] 24.P1 RPG: maximum monsters defeated
- [ ] 24.P2 Andy's vacation (cancel ≤ K obligations)
- [ ] 24.P3 Gym: fewest exercises to drain energy
- [ ] 24.P4 GfG · Wine Buying and Selling
- [ ] 24.P5 Arrays where each element divides the next (mod 10000)

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 28, Consolidate.

DAY 28 OF 29 · FRIDAY 4 DECEMBER 2026 · LEARNING DAY · PHASE 4 · EXAM SIMULATION

28

Consolidate

Templates from memory, a speed drill, and error-log re-solves. No new topics.

Read first · Theory Chapter 28
Patterns · —

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	Read Theory Chapter 28; type each new listing once and run it	40 min
☐	28.P1 LC 217 · Contains Duplicate	15 min
☐	28.P2 LC 881 · Boats to Save People	15 min
☐	28.P3 LC 2300 · Successful Pairs of Spells and Potions	15 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
	Core total (estimate)	**3.3 h**

NOTES FOR THE DAY

- Handbook in recall mode: write every template from memory, timed. Target: under 40 minutes for the 15 you rated weakest.
- Re-solve 4 error-log problems.
- Speed drill: the 3 problems below, each in under 15 minutes.
- Read the exam-day playbook. Stop by 8 pm.

Why it matters for Infosys. Writing templates quickly and without errors is what saves time on the real Q1.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Lazy segment tree (range add, range max)	Ch. 21	7 days ago	☐ Again ☐ Shaky ☐ Solid
Split into K parts (dp[k][i])	Ch. 21	7 days ago	☐ Again ☐ Shaky ☐ Solid
Interval DP and game DP	Ch. 14	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 3 core problems, 0 bonus.

28.P1 LC 217 · Contains Duplicate EASY CORE — **15 min**

`leetcode.com/problems/contains-duplicate` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 28.P1*

28.P2 LC 881 · Boats to Save People MEDIUM CORE — **15 min**

`leetcode.com/problems/boats-to-save-people` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 28.P2*

28.P3 LC 2300 · Successful Pairs of Spells and Potions MEDIUM CORE — **15 min**

`leetcode.com/problems/successful-pairs-of-spells-and-potions` · LeetCode

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 28.P3*

Re-solve from three days ago

Pick one problem from Day 25 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 25.P1 Sample paper · Easy (tree beauty) ☐ 25.P2 Sample paper · Medium (gcd under updates)

☐ 25.P3 Sample paper · Hard (LNDS with XOR ≥ M)

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow: Chapter 29, Rest and logistics.

DAY 29 OF 29 · SATURDAY 5 DECEMBER 2026 · REST DAY · PHASE 4 · EXAM SIMULATION

29

Rest and logistics

One warm-up, pack, plan the route, sleep.

Read first · Theory Chapter 29
Patterns · —

Today's plan

	Block	Time
☐	Handbook review: recall the patterns listed below from memory	15 min
☐	29.P1 LC 1480 · Running Sum of 1d Array	5 min
☐	Review: error log, re-solve one problem from 3 days ago	20 min
	Core total (estimate)	**1.2 h**

NOTES FOR THE DAY

- One easy warm-up (below) so the workflow feels routine.
- Pack exactly what the Infosys invite lists. Confirm the venue and reporting time.
- Plan to leave early. Lay out tomorrow's things tonight.
- Sleep 7–8 hours. Don't solve problems late at night.

Why it matters for Infosys. You have done the work. A rested brain is worth more on Q3 than one more problem tonight.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

Pattern	Taught	Interval	Rating
Backtracking and subset enumeration	Ch. 22	7 days ago	☐ Again ☐ Shaky ☐ Solid
Bitmask DP	Ch. 22	7 days ago	☐ Again ☐ Shaky ☐ Solid
Digit DP	Ch. 22	7 days ago	☐ Again ☐ Shaky ☐ Solid
BFS: fewest edges, grids, many sources	Ch. 15	14 days ago	☐ Again ☐ Shaky ☐ Solid
DFS without recursion, components	Ch. 15	14 days ago	☐ Again ☐ Shaky ☐ Solid
Topological order (Kahn) and DAG DP	Ch. 15	14 days ago	☐ Again ☐ Shaky ☐ Solid
0-1 BFS	Ch. 15	14 days ago	☐ Again ☐ Shaky ☐ Solid

Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 1 core problems, 0 bonus.

29.P1 LC 1480 · Running Sum of 1d Array EASY CORE — **5 min**

`leetcode.com/problems/running-sum-of-1d-array · LeetCode`

Pattern	Target O( )
Key idea	
Edge cases tested	

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 29.P1*

Re-solve from three days ago

Pick one problem from Day 26 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 26.P1 Max XOR choosing at most N/2 elements ☐ 26.P2 Pairs with XOR < K and a prime sum

End of day

Problem	What went wrong	Cause	Rule for next time	Re-solve on

One thing I understand today that I did not understand yesterday:

One thing that is still shaky (re-read it in the theory book tomorrow):

Tomorrow is the exam.

APPENDIX A

Hints

One hint per problem, taken from the 29-day plan. Open it only when the time limit on the worksheet has run out.

Day 1 · Set up, Java toolkit, fast input

1.P1 **LC 1 · Two Sum.** One pass: before storing a[i], look for target − a[i] in the map.

1.P2 **LC 242 · Valid Anagram.** Count letters in an int[26]; one string adds, the other subtracts.

1.P3 **LC 20 · Valid Parentheses.** ArrayDeque as a stack: push the closer you expect, pop and compare.

1.P4 **LC 347 · Top K Frequent Elements.** HashMap of counts, then a min-heap of size k (or buckets by count).

1.P5 **LC 729 · My Calendar I.** TreeMap start → end. Check floorKey(start) and ceilingKey(start) for overlap.

1.P6 **LC 206 · Reverse Linked List.** Three pointers: prev, cur, next.

1.P7 **LC 141 · Linked List Cycle.** Slow moves 1, fast moves 2; they meet only if there is a cycle.

1.P8 **LC 49 · Group Anagrams.** Key = the sorted letters of the word.

Day 2 · Prefix sums, difference arrays, hashing

2.P1 **LC 560 · Subarray Sum Equals K.** Prefix + HashMap seeded with {0: 1}.

2.P2 **LC 974 · Subarray Sums Divisible by K.** Key = Math.floorMod(prefix, k); equal remainders make a valid subarray.

2.P3 **LC 1524 · Number of Sub-arrays With Odd Sum.** Count odd and even prefixes seen so far.

2.P4 **LC 525 · Contiguous Array.** Treat 0 as −1; store the FIRST index of each prefix value.

2.P5 **LC 1109 · Corporate Flight Bookings.** Difference array.

2.P6 **LC 304 · Range Sum Query 2D - Immutable.** 2D prefix: P[i+1][j+1] = P[i][j+1] + P[i+1][j] − P[i][j] + a[i][j].

2.P7 **CF 466C · Number of Ways.** Prefix sums: count split points for one third of the total, then for two thirds.

Day 3 · Two pointers and sliding window

3.P1 **LC 3 · Longest Substring Without Repeating Characters.** Window with a count (or last-seen index) per character.

3.P2 **LC 1004 · Max Consecutive Ones III.** Window with at most K zeros: the vacation pattern.

3.P3 **LC 424 · Longest Repeating Character Replacement.** Valid while window length − (most frequent count) ≤ k.

3.P4 **LC 15 · 3Sum.** Sort, fix i, two pointers on the rest, skip duplicates.

3.P5 **LC 992 · Subarrays with K Different Integers.** Exactly K = atMost(K) − atMost(K − 1).

3.P6 **LC 239 · Sliding Window Maximum.** Monotonic deque of indices.

3.P7 **LC 76 · Minimum Window Substring.** Counts of what you need vs what the window has; shrink while valid.

Day 4 · Sorting and greedy: intervals

4.P1 **LC 56 · Merge Intervals.** Sort by start; extend the last interval while they overlap.

4.P2 **LC 435 · Non-overlapping Intervals.** Sort by end; answer = n − (most you can keep).

4.P3 **LC 2406 · Divide Intervals Into Minimum Number of Groups.** This is "minimum platforms": sweep sorted starts and ends.

4.P4 **LC 179 · Largest Number.** Sort strings with (b + a).compareTo(a + b). All zeros → "0".

4.P5 **LC 1798 · Maximum Number of Consecutive Values You Can Make.** Sort; if a coin > reach + 1, stop, else reach += coin.

4.P6 **LC 452 · Minimum Number of Arrows to Burst Balloons.** Sort by end; touching balloons share an arrow.

4.P7 **LC 135 · Candy.** Two passes: left to right, then right to left.

Day 5 · Greedy with heaps

5.P1 **LC 502 · IPO.** Sort projects by capital; heap of profits you can afford.

5.P2 **LC 1642 · Furthest Building You Can Reach.** Min-heap of the climbs given ladders; the smallest pays bricks.

5.P3 **LC 1526 · Minimum Number of Increments on Subarrays to Form a Target Array.** Answer = sum of the positive rises a[i] − a[i−1].

5.P4 **LC 1046 · Last Stone Weight.** Max-heap: poll two, push the difference.

5.P5 **LC 871 · Minimum Number of Refueling Stops.** Heap of fuel at stations you have passed; refuel from the biggest when stuck.

5.P6 **LC 630 · Course Schedule III.** Sort by deadline; max-heap of durations; drop the longest when over time.

5.P7 **CF 1526C2 · Potions (Hard Version).** Drink every potion; when health goes negative, undo the most negative one so far.

Day 6 · Binary search, including on the answer

6.P1 **LC 34 · Find First and Last Position of Element in Sorted Array.** Lower bound of x, and lower bound of x + 1, minus 1.

6.P2 **LC 875 · Koko Eating Bananas.** Smallest speed with total hours ≤ h. Hours for a pile = ceil(pile / speed).

6.P3 **LC 1011 · Capacity To Ship Packages Within D Days.** Smallest capacity whose greedy day count ≤ days.

6.P4 **LC 410 · Split Array Largest Sum.** Same check as ship capacity: count parts greedily.

6.P5 **LC 1552 · Magnetic Force Between Two Balls.** Largest gap that still places m balls (round mid up).

6.P6 **LC 2226 · Maximum Candies Allocated to K Children.** Largest amount x with Σ pile / x ≥ k.

6.P7 **LC 719 · Find K-th Smallest Pair Distance.** Binary search the distance; count pairs ≤ d with two pointers.

Day 7 · Monotonic stacks

7.P1 **LC 739 · Daily Temperatures.** Next greater element to the right; answer = index difference.

7.P2 **LC 503 · Next Greater Element II.** Circular: loop i from 0 to 2n − 1 and use i % n.

7.P3 **LC 84 · Largest Rectangle in Histogram.** Previous and next smaller bar give the width for each bar.

7.P4 **LC 907 · Sum of Subarray Minimums.** Contribution counting, then mod 10^9+7.

7.P5 **LC 402 · Remove K Digits.** Greedy stack: pop while the top is bigger than the current digit.

7.P6 **LC 316 · Remove Duplicate Letters.** Stack + last index of each letter + "already in stack" flags.

7.P7 **LC 2104 · Sum of Subarray Ranges.** Sum of maximums minus sum of minimums, both by contribution.

Day 8 · Bits and XOR

8.P1 **LC 136 · Single Number.** XOR everything; pairs cancel.

8.P2 **LC 1442 · Count Triplets That Can Form Two Arrays of Equal XOR.** a == b means xor(i..k) == 0: equal prefix XORs.

8.P3 **LC 421 · Maximum XOR of Two Numbers in an Array.** Binary trie; for each x, walk towards the opposite bits.

8.P4 **LC 1803 · Count Pairs With XOR in a Range.** Trie with counts: pairs with XOR < high + 1 minus pairs with XOR < low.

8.P5 **LC 2429 · Minimize XOR.** Greedy on bits: match num1's high bits first, then fill low bits.

8.P6 **GfG · Maximum Subset XOR.** XOR basis (Gaussian elimination over bits).

8.P7 **LC 1707 · Maximum XOR With an Element From Array.** Offline: sort queries by limit, insert numbers into the trie as they qualify.

Day 10 · Number theory toolkit

10.P1 **LC 204 · Count Primes.** Sieve of Eratosthenes up to n − 1.

10.P2 **LC 1922 · Count Good Numbers.** 5^(even positions) × 4^(odd positions) with fast power mod 10^9+7.

10.P3 **LC 2447 · Number of Subarrays With GCD Equal to K.** For each start, extend while gcd is a multiple of k; stop early otherwise.

10.P4 **LC 2862 · Maximum Element-Sum of a Complete Subset of Indices.** Group indices by square-free kernel; the best group sum wins.

10.P5 **LC 1819 · Number of Different Subsequences GCDs.** For each g, gcd of all multiples of g present in the array; count when it equals g.

10.P6 **LC 2572 · Count the Number of Square-Free Subsets.** Numbers ≤ 30: bitmask of the 10 primes, knapsack over masks.

10.P7 **CF 1497E1 · Square-Free Division (easy version).** Square-free kernel, then greedy cuts when a kernel repeats.

Day 11 · DP I: states, transitions, knapsack

11.P1 **LC 198 · House Robber.** Take or skip: best[i] = max(best[i−1], best[i−2] + a[i−1]).

11.P2 **LC 322 · Coin Change.** dp[a] = min over coins of dp[a − c] + 1; start with a large INF.

11.P3 **LC 518 · Coin Change II.** Combinations: coins in the outer loop, amount in the inner loop.

11.P4 **LC 416 · Partition Equal Subset Sum.** Boolean subset sum to total / 2, looping the sum downwards.

11.P5 **LC 494 · Target Sum.** Count subsets with sum (total + target) / 2.

11.P6 **LC 1049 · Last Stone Weight II.** Split into two groups as equal as possible: subset sum up to total / 2.

11.P7 **LC 740 · Delete and Earn.** Sum points per value, then House Robber over values.

Day 12 · DP II: two-string DP

12.P1 **LC 1143 · Longest Common Subsequence.** Match → diagonal + 1, else the better of up and left.

12.P2 **LC 72 · Edit Distance.** Insert, delete, replace = left, up, diagonal + 1.

12.P3 **LC 516 · Longest Palindromic Subsequence.** dp[l][r] on one string, or LCS with its reverse.

12.P4 **LC 1312 · Minimum Insertion Steps to Make a String Palindrome.** n − longest palindromic subsequence.

12.P5 **LC 115 · Distinct Subsequences.** dp[i][j] = dp[i−1][j] + (match ? dp[i−1][j−1] : 0). Use long.

12.P6 **LC 97 · Interleaving String.** dp[i][j] = can the first i of s1 and j of s2 form the first i + j of s3.

12.P7 **LC 1092 · Shortest Common Supersequence.** Build the LCS table, then walk back to print.

Day 13 · DP III: LIS and DP with an extra dimension

13.P1 **LC 300 · Longest Increasing Subsequence.** Write the O(n log n) tails version.

13.P2 **LC 1964 · Find the Longest Valid Obstacle Course at Each Position.** Non-decreasing LIS: upper bound; the answer at i is the position + 1.

13.P3 **LC 1671 · Minimum Number of Removals to Make Mountain Array.** LIS ending at i from the left and from the right; peak needs both ≥ 2.

13.P4 **LC 2370 · Longest Ideal Subsequence.** dp over the last letter: 26 states.

13.P5 **LC 354 · Russian Doll Envelopes.** Sort by width ascending, height descending; LIS on heights.

13.P6 **LC 673 · Number of Longest Increasing Subsequence.** For each i, keep (length, count).

Day 14 · DP IV: divisor, interval and game DP

14.P1 **LC 368 · Largest Divisible Subset.** Sort; LIS-style DP where j → i is allowed if a[i] % a[j] == 0.

14.P2 **LC 486 · Predict the Winner.** dp[l][r] = max(a[l] − dp[l+1][r], a[r] − dp[l][r−1]).

14.P3 **LC 1547 · Minimum Cost to Cut a Stick.** Add 0 and n as cuts; interval DP over cut indices.

14.P4 **LC 1690 · Stone Game VII.** Same game DP; the score of a move is the sum of what is left (prefix sums).

14.P5 **LC 2338 · Count the Number of Ideal Arrays.** Divisor chains of s distinct values × C(n − 1, s − 1).

14.P6 **LC 312 · Burst Balloons.** Choose the LAST balloon to burst in (l, r).

14.P7 **LC 664 · Strange Printer.** dp[l][r]; when s[k] == s[l], the two prints can merge.

Day 15 · Graphs I: BFS, DFS, topological order

15.P1 **LC 200 · Number of Islands.** Each unvisited land cell starts a BFS/DFS that marks its island.

15.P2 **LC 994 · Rotting Oranges.** Multi-source BFS from every rotten orange; the answer is the last layer.

15.P3 **LC 207 · Course Schedule.** Kahn's algorithm; possible iff every course gets processed.

15.P4 **LC 1091 · Shortest Path in Binary Matrix.** BFS with 8 directions; distance counts cells, so start at 1.

15.P5 **LC 785 · Is Graph Bipartite?.** BFS colouring; a neighbour with the same colour means no.

15.P6 **LC 1368 · Minimum Cost to Make at Least One Valid Path in a Grid.** 0-1 BFS: following the arrow costs 0, changing it costs 1.

15.P7 **LC 2050 · Parallel Courses III.** Topological order; finish[v] = time[v] + max finish of prerequisites.

Day 17 · Graphs II: Dijkstra, DSU, MST

17.P1 **LC 743 · Network Delay Time.** Dijkstra from k; the answer is the largest distance (−1 if any is unreachable).

17.P2 **LC 1631 · Path With Minimum Effort.** Dijkstra where a path costs its largest step.

17.P3 **LC 1584 · Min Cost to Connect All Points.** MST over all pairs (Kruskal, or Prim in $O(n^2)$).

17.P4 **LC 1976 · Number of Ways to Arrive at Destination.** Dijkstra plus ways[]; distances need long.

17.P5 **LC 787 · Cheapest Flights Within K Stops.** State = (city, stops), or Bellman-Ford for k + 1 rounds.

17.P6 **LC 2290 · Minimum Obstacle Removal to Reach Corner.** 0-1 BFS: an obstacle cell costs 1.

17.P7 **LC 2421 · Number of Good Paths.** Process nodes by value; DSU joins neighbours with value ≤ current.

Day 18 · Trees I: recursion, parent arrays, LCA

18.P1 **LC 543 · Diameter of Binary Tree.** Height of each side; the diameter bends at some node.

18.P2 **LC 236 · Lowest Common Ancestor of a Binary Tree.** Return the node if it is p or q; if both sides return something, this is the LCA.

18.P3 **LC 124 · Binary Tree Maximum Path Sum.** Return the best downward sum; drop negative branches.

18.P4 **LC 2246 · Longest Path With Different Adjacent Characters.** Parent array input: reverse BFS order, keep the two best child chains.

18.P5 **LC 102 · Binary Tree Level Order Traversal.** BFS; process q.size() nodes per level.

18.P6 **LC 1483 · Kth Ancestor of a Tree Node.** Binary lifting table.

18.P7 **LC 2477 · Minimum Fuel Cost to Report to the Capital.** Subtree sizes: each edge needs ceil(size / seats) cars.

18.P8 **LC 2846 · Minimum Edge Weight Equilibrium Queries in a Tree.** LCA plus counts of each weight (1..26) on the root path.

Day 19 · Trees II: tree DP, rerooting, small-to-large

19.P1 **LC 337 · House Robber III.** Each node returns {best if robbed, best if not}.

19.P2 **LC 1519 · Number of Nodes in the Sub-Tree With the Same Label.** Count labels per subtree: int[26] per node, merged from children (edges given: root it first).

19.P3 **LC 834 · Sum of Distances in Tree.** Rerooting: ans[child] = ans[parent] − size[child] + (n − size[child]).

19.P4 **LC 3068 · Find the Maximum Sum of Node Values.** Any even number of nodes can be XORed; greedy by gain, or dp[v][parity].

19.P5 **LC 2858 · Minimum Edge Reversals So Every Node Is Reachable.** Rerooting over edge directions: reversals from 0, then adjust per edge.

19.P6 **LC 2003 · Smallest Missing Genetic Value in Each Subtree.** Only the path from the node with value 1 to the root matters; or small-to-large sets.

19.P7 **CF 600E · Lomsat gelral.** Small-to-large maps of colour counts, tracking the maximum count and its sum.

Day 20 · Fenwick trees and segment trees

20.P1 **LC 307 · Range Sum Query - Mutable.** Fenwick or segment tree; update stores the difference.

20.P2 **LC 315 · Count of Smaller Numbers After Self.** Compress values; scan from the right, query the Fenwick tree, then add.

20.P3 **LC 2407 · Longest Increasing Subsequence II.** Segment tree of max over values; query [x − k, x − 1].

20.P4 **CF 914D · Bash and a Tough Math Puzzle.** Gcd segment tree; count how many positions break the gcd (stop after 2).

20.P5 **LC 2286 · Booking Concert Tickets in Groups.** Segment tree of max and sum; descend to the first row with room.

20.P6 **LC 3165 · Maximum Sum of Subsequence With Non-adjacent Elements.** Each node stores 4 values: best with or without its two ends.

20.P7 **LC 2322 · Minimum Score After Removals on a Tree.** Euler tour times decide whether one removed subtree contains the other.

Day 21 · Lazy segment trees and the K-parts DP

21.P1 **LC 1335 · Minimum Difficulty of a Job Schedule.** dp[day][i] over the last day's start; cost = max of that part.

21.P2 **LC 813 · Largest Sum of Averages.** dp[k][i] with prefix sums for the average of a part.

21.P3 **LC 2916 · Subarrays Distinct Element Sum of Squares II.** Lazy segment tree over start positions: a new element adds 1 to a range of starts.

21.P4 **CF 833B · The Bakery.** The Infosys "gift boxes": dp[k][i] with a lazy max segment tree (handbook card).

21.P5 **LC 732 · My Calendar III.** Lazy segment tree (range add, global max), or a TreeMap sweep.

21.P6 **CF 1527E · Partition Game.** Infosys "k segments min cost": range add of i − prev[a[i]] over earlier split points.

Day 22 · Backtracking, bitmask DP, digit DP

22.P1 **LC 78 · Subsets.** Bitmask loop, or choose / skip recursion.

22.P2 **LC 1986 · Minimum Number of Work Sessions to Finish the Tasks.** dp[mask] = (sessions, time used in the last one), compared in that order.

22.P3 **LC 1879 · Minimum XOR Sum of Two Arrays.** Assignment with dp[mask]: the next element of nums1 picks an unused element of nums2.

22.P4 **LC 902 · Numbers At Most N Given Digit Set.** Shorter lengths: |D|^len. Same length: digit by digit with tight.

22.P5 **LC 51 · N-Queens.** Backtracking with column and two diagonal flags.

22.P6 **LC 847 · Shortest Path Visiting All Nodes.** BFS over (node, visited mask).

22.P7 **LC 2827 · Number of Beautiful Integers in the Range.** Digit DP: (pos, even − odd, remainder mod k, tight, started).

Day 24 · Infosys repeat problems I

24.P1 **RPG: maximum monsters defeated.** Sort by power; walk while experience ≥ power.

24.P2 **Andy's vacation (cancel ≤ K obligations).** Count obligations per day; sliding window over days.

24.P3 **Gym: fewest exercises to drain energy.** Sort drains descending; use each twice.

24.P4 **GfG · Wine Buying and Selling.** Answer = sum of |prefix sum|.

24.P5 **Arrays where each element divides the next (mod 10000).** Divisor chains × Pascal's triangle mod 10000.

24.P6 **LIS with (x & y)·2 < (x | y).** The condition means the highest set bits differ.

24.P7 **Smallest base with identical digits.** Try bases ≤ 10^6, then the two-digit case M = d·(B + 1).

24.P8 **Choose X ≤ K maximising Σ X xor A[i].** Per-bit greedy under K, like digit DP.

24.P9 **CF 1338A · Powered Addition.** The idea behind Infosys "terrain days".

24.P10 **CF 1514C · Product 1 Modulo N.** Infosys "product 1 mod N".

Day 26 · Repair day + the hard XOR repeat

26.P1 **Max XOR choosing at most N/2 elements.** XOR knapsack: fewest elements for each XOR value below 2^{20}.

26.P2 **Pairs with XOR < K and a prime sum.** Sieve once to 2·max(A); trie for many pairs.

26.P3 **Sample Complex: brute force for N ≤ 15.** Enumerate matchings for N ≤ 15, check every root-to-leaf path.

Day 28 · Consolidate

28.P1 **LC 217 · Contains Duplicate.** Speed drill: HashSet.

28.P2 **LC 881 · Boats to Save People.** Speed drill: sort + two pointers.

28.P3 **LC 2300 · Successful Pairs of Spells and Potions.** Speed drill: sort + lower bound.

Day 29 · Rest and logistics

29.P1 **LC 1480 · Running Sum of 1d Array.** 5-minute warm-up.

APPENDIX B

Answer key

Answers to the exercises. Check your work after you finish a section, not during it.

Chapter 1 · Java for Problem Solving: Collections, Input and Output

Exercise 1.1. (1) `st == null || !st.hasMoreTokens()`; (2) `return null`; (3) `new StringTokenizer(line)`; (4) `fs.nextLong()`; (5) `out.flush()`. The `while` loop in `next()` keeps reading lines until the tokenizer has a word, so a line break is treated like a space. Whether the four numbers share a line or have one line each, `next()` returns them in the same order.

Exercise 1.2.

char	action	stack (bottom → top)
(	push)	)
[	push]	)]
]	pop], matches	)
{	push }	)}
(	push)	)})
)	pop), matches	)}
}	pop }, matches	)
)	pop), matches	(empty)
]	stack is empty: return `false`	(empty)

The `expect.isEmpty()` half of the condition returns `false`: a closer arrived when no opener was waiting. Without the last character the loop ends with an empty stack and the method returns `true`.

Exercise 1.3. [12, 15): floor 8 (12), ceiling none, accepted (touching 12 is allowed). [10, 13): floor 8 (12), ceiling 12 (15); rejected because [8, 12) ends after 10. [0, 3): floor none, ceiling 3 (8), accepted because 3 is not less than 3. [15, 16): floor 12 (15), ceiling none, accepted. [14, 16): floor 12 (15), ceiling 15 (16); rejected because [12, 15) ends after 14. Final map: {0: 3, 3: 8, 8: 12, 12: 15, 15: 16}.

Exercise 1.4. (1) −2,147,479,015: the true product 2,147,488,281 exceeds $2^{31} - 1$ and wraps by 2^{32}. (2) −2 and 2: `%` takes the sign of the left operand, `floorMod` does not. (3) `true` and `false`: 127 is in the `Integer` cache −128..127, 200 is not, so two different objects are created. (4) `false` and `true`: `t` is built at run time, so it is a new object with the same text. (5) [5, 7]: `remove(1)` with an `int` argument removes index 1; `remove(Integer.valueOf(1))` would remove a value. (6) It throws a `NullPointerException`: `ArrayDeque` refuses `null`. (7) 195 (an `int`: 97 + 98) and `'z'`. (8) False: `toString` shows the internal heap array; only the first element is guaranteed to be the smallest.

Exercise 1.5. (1) $2 \cdot 10^5 \cdot (2 \cdot 10^5 - 1)/2 = 19{,}999{,}900{,}000 \approx 2 \cdot 10^{10}$ pairs: far too slow. (2) $2^{20} \cdot 20 = 20{,}971{,}520 \approx 2 \cdot 10^7$ steps: fits. (3) About $10^5 \cdot \log_2 10^5 \approx 1.7 \cdot 10^6$ for the sort and as much again for the searches: fits easily. (4) $3000^3 = 2.7 \cdot 10^{10}$ triples (or about $4.5 \cdot 10^9$ if only $i < j < k$): too slow; $n \le 500$ is the usual limit for $O(n^3)$. (5) $25 \cdot 10^6$ cells × 8 bytes = $2 \cdot 10^8$ bytes, about 200 MB: too much for most limits.

Exercise 1.6. Keep the free spots in a `TreeSet<Integer>`, initially $1..m$. "Arrive p" is `ceiling(p)`: if it is `null`, print −1; otherwise remove that spot and print it. "Leave s" is `add(s)`. Each event costs $O(\log m)$, so the total is $O(m \log m + q \log m)$. For the events given the arrivals print 2, 3, 5, 6, 3, −1, 1, 4. (After `leave 3` the free spots are {1, 3, 4}; the arrival preferring 6 finds no free spot ≥ 6; after `leave 6` the arrival preferring 4 takes 4.)

Exercise 1.7. `cnt.get(x) == cnt.get(y)` compares two `Integer` objects by reference. Counts up to 127 come from the `Integer` cache, so equal counts are the same object and the test passes. With $x = 1$ and $y = 2$ each occurring 128 times the method returns `false` (it returns `true` with 127 each). Fix: compare values, for example `return cnt.getOrDefault(x, 0).intValue() == cnt.getOrDefault(y, 0).intValue();`, which also handles a value that does not occur at all.

Exercise 1.8. Compare your code with *Theory Listing 1.8* line by line, especially the order `nxt = head.next; head.next = prev; prev = head; head = nxt;` and the check `fast != null && fast.next != null`. The meeting step is the smallest multiple of λ that is at least max(μ, 1): for μ = 4, λ = 6 it is 6; for μ = 10, λ = 3 it is 12.

Chapter 2 · Prefix Sums, Difference Arrays and Hashing

Exercise 2.1. (1) `seen.put(0L, 1);` (2) `seen.getOrDefault(prefix - k, 0);` (3) `seen.merge(prefix, 1, Integer::sum)`. The look-up must come first so that the current prefix is never matched with itself; otherwise, for $k = 0$, every position adds a phantom empty subarray.

Exercise 2.2.

i	a[i]	prefix	prefix − k	found	total
0	3	3	−4	0	0
1	4	7	0	1	1
2	−7	0	−7	0	1
3	3	3	−4	0	1
4	1	4	−3	0	1
5	3	7	0	2	3
6	1	8	1	0	3
7	−4	4	−3	0	3
8	−2	2	−5	0	3
9	−2	0	−7	0	3

The answer is 3. At row 5 the value 0 has been seen twice (the empty prefix and the prefix after index 2), so two subarrays end there: $a[0..5] = 3 + 4 - 7 + 3 + 1 + 3$ and $a[3..5] = 3 + 1 + 3$. The third is $a[0..1] = 3 + 4$, found at row 1.

Exercise 2.3. Rows 3–5: remainders 2, 4, 0; counts found before 0, 3, 1; totals 3, 6, 7; `cnt` after row 5 is [2, 0, 1, 0, 4]. The answer is 7, which matches LeetCode 974's first example. With plain `%`, row 4 computes (2 + (−3)) % 5 = −1, and `cnt[-1]` throws an `ArrayIndexOutOfBoundsException`. `Math.floorMod(-1, 5)` is 4, the correct key.

Exercise 2.4. $d = [2, -1, 5, -5, 1, 0, -2]$ and $a = [2, 1, 6, 1, 2, 2]$. Check $a[2]$: +2 (from [0..3]), +5, −1, so 6.

Exercise 2.5. S rows 1–3 are [0, 3, 3, 4], [0, 8, 14, 18], [0, 9, 17, 21]. The query is 21 − 4 − 9 + 3 = 11, and directly 6 + 3 + 2 + 0 = 11.

Exercise 2.6. (1) True: with positive values the window sum is monotone in each pointer (Chapter 3). (2) False: store the *first* index; later indices give shorter subarrays. (3) False: an `int[]` of size k is faster and has no boxing or worst case. (4) False: $a[4]$ is a prefix sum of d, which costs $O(n)$ to recompute; use a Fenwick tree. (5) True: a subarray's sum is odd exactly when its two end prefixes have different parity.

Exercise 2.7. Map 0 → −1 and 1 → +1; a substring is balanced when its sum is 0, so its two end prefixes are equal. Store each prefix value with a **count**, look up the **same** value, seed {0: 1}. For `0011010` the prefixes are 0, −1, −2, −1, 0, −1, 0, −1. The value −1 occurs 4 times and 0 occurs 3 times, so the answer is C(4, 2) + C(3, 2) = 6 + 3 = 9.

Exercise 2.8. Negative elements break it. With $a = [-1, 1]$ and $k = 5$ the prefix remainder becomes −1, the loop stops, and the method returns 0 instead of 1 (the subarray [−1, 1] has sum 0). Fix: `prefix = Math.floorMod(prefix + x, k);` and delete the `if`.

Chapter 3 · Two Pointers and Sliding Windows

Exercise 3.1. (1) `cnt.merge(a[r], 1, Integer::sum)`; (2) `cnt.size() > k`; (3) `cnt.remove(a[l])`; (4) `l++`; (5) `Math.max(best, r - l + 1)`. To count all valid windows, replace line (5) by `count += r - l + 1` (with a `long count`), because every window $[l', r]$ with $l \le l' \le r$ is valid.

Exercise 3.2.

i	a[i]	removed at back	removed at front	deque (indices)	max
0	3	—	—	0	—
1	1	—	—	0 1	—
2	5	1, 0	—	2	5
3	2	—	—	2 3	5
4	2	3	—	2 4	5
5	6	4, 2	—	5	6
6	0	—	—	5 6	6
7	4	6	—	5 7	6

The output is [5, 5, 5, 6, 6, 6]. In this input no index ever leaves at the front, because each front is removed at the back by a larger value before it expires. At $i = 4$ the `<=` removes index 3: the newer copy at index 4 has the same value and stays in the window longer, so the older copy can never be needed.

Exercise 3.3. $r = 3$: l 0 → 2, counts {1: 1, 3: 1}, length 2, best 3. $r = 4$: l stays 2, {1: 1, 3: 2}, length 3, best 3. $r = 5$: l 2 → 3, {2: 1, 3: 2}, length 3, best 3. $r = 6$: l stays 3, {2: 2, 3: 2}, length 4, best 4. $r = 7$: l 3 → 5, {2: 2, 4: 1}, length 3, best 4. The answer is 4, the window [3, 3, 2, 2].

Exercise 3.4. (1) Monotone: a sub-window has no more zeros; grow-and-shrink. (2) Not monotone: [4, 6] is valid but its sub-window [4] is not. Extending a valid window keeps it valid, so shrink-while-valid finds the shortest. (3) Not monotone ([4, 6] is valid, [4] is not) and not closed under extension ([4, 6, 1] is invalid). Neither loop works directly; count it as (windows with sum ≤ 10) − (windows with sum ≤ 9), two monotone scans, or use prefix sums. (4) Monotone: a sub-window's maximum is no larger and its minimum no smaller; grow-and-shrink (with two deques, Exercise 3.6). (5) Not monotone, but closed under extension; shrink-while-valid with need/have counters. (6) Not monotone: [20, −15] has sum 5, its sub-window [20] has sum 20; neither loop, use prefix sums.

Exercise 3.5. atMost(2): l = 0, 0, 0, 2, 2 and additions 1, 2, 3, 2, 3, total 11. atMost(1): l = 0, 0, 2, 3, 4 and additions 1, 2, 1, 1, 1, total 6. Exactly two distinct values: 11 − 6 = 5. They are [2, 2, 1], [2, 1], [1, 3], [1, 3, 1] and [3, 1].

Exercise 3.6. Removing an end element cannot raise the maximum or lower the minimum, so sub-windows of valid windows are valid: grow-and-shrink applies. Keep two monotonic deques of indices: one with decreasing values (its front is the window maximum) and one with increasing values (its front is the minimum). When $a[r]$ enters, pop dominated indices from the back of each and append r. While front-max − front-min > d, advance l and pop the front of either deque if its index is now < l. Each index enters and leaves each deque once, so the total is $O(n)$. For the example the answer is 3, for instance [4, 6, 5] or [9, 7, 8].

Exercise 3.7. The method never removes a key whose count drops to 0, so `cnt.size()` never decreases. As soon as the array contains more than k distinct values, the `while` loop keeps advancing l past r and off the end of the array. For a = [1, 2, 3] and $k = 2$ it throws `ArrayIndexOutOfBoundsException` at $r = 2$. The samples it passed, such as [1, 2, 1, 2] and [5, 5, 5] with $k = 2$, never exceed k distinct values. Fix: `if (cnt.merge(a[l], -1, Integer::sum) == 0) cnt.remove(a[l]);`.

Exercise 3.8. (1) Negative values break windows: prefix sums with a hash map, $O(n)$ expected. (2) All values positive, so "sum ≥ S" survives extension: shrink-while-valid window, $O(n)$. (3) Order does not matter: sort, fix i, then count pairs with sum < $T - a[i]$ using `countPairsAtMost` on the suffix; $O(n^2) \approx 4.5 \cdot 10^6$ steps. (4) Fixed window size:

monotonic deque with the comparison flipped, O(n). (5) "At most 3 distinct" is monotone: grow-and-shrink with an `int[]` of counts, O(n).

Exercise 3.9. Compare with *Theory Listing 3.9*. Invariant: the deque holds indices of the current window in increasing order with strictly decreasing values, and every window index not in it is dominated by a later, at least as large index. Each index is added once and removed at most once, so there are at most 2n deque operations.

Chapter 4 · Sorting and Greedy Choice: Intervals

Exercise 4.1. (1) `Integer.compare(x[0], y[0]);` (2) `cur[0] <= last[1];` (3) `Math.max(last[1], cur[1]);` (4) `Integer.compare(x[1], y[1]);` (5) `cur[0] >= end`. Merging needs every interval that can touch the current block to arrive while that block is the last one, which sorting by start guarantees. Scheduling needs the choice that leaves the most room, which is the earliest end (*Theory Theorem 4.3*). `merge` returns [1, 3] for [1, 2], [2, 3], because touching intervals merge.

Exercise 4.2. The events, in order, with busy after each: 1 start (1), 2 start (2), 3 start (3), 3 end (2), 4 start (3), 4 end (2), 5 end (1), 6 start (2), 6 end (1), 7 start (2), 8 end (1), 9 end (0). The best is 3, so 3 rooms are needed (for example at time 3, meetings [1, 4], [2, 3] and [3, 5] all contain 3). If a room freed at t can be reused at t, ends sort first and the maximum drops to 2.

Exercise 4.3. [3, 5]: the earliest-freed room ends at 3, which is not < 3, so a new room; heap {3, 4, 5}. [4, 6]: reuse the room freed at 3; {4, 5, 6}. [6, 8]: reuse the room freed at 4; {5, 6, 8}. [7, 9]: reuse the room freed at 5; {6, 8, 9}. Three rooms, matching Exercise 4.2.

Exercise 4.4. Take any order with neighbours i before j and $t_i > t_j$, and let T be the time when the first of them starts. Every other job finishes at the same time in both orders. In the given order the two finish at $T + t_i$ and $T + t_i + t_j$; after swapping, at $T + t_j$ and $T + t_j + t_i$. The sum changes by $t_j - t_i < 0$, so the swap strictly improves it. An optimal order therefore has no such pair, which means it is sorted by t (and ties do not matter). For t = (3, 1, 2) the six orders give 3 1 2 → 13, 3 2 1 → 14, 1 3 2 → 11, 1 2 3 → 10, 2 3 1 → 13, 2 1 3 → 11; the sorted order 1, 2, 3 gives the minimum 10.

Exercise 4.5. Amount 6: the greedy takes 4 + 1 + 1 (three coins), but 3 + 3 uses two. (Amount 10 also fails: 4 + 4 + 1 + 1 against 4 + 3 + 3.) For coins {1, 5, 10, 25} the rule agreed with an exact dynamic programme (Chapter 11) for every amount up to 500. That is the way to gain confidence: compare the greedy with an exact method on every small input, which still is evidence, not proof.

Exercise 4.6. The order is 9, 89, 898, 8, 0, giving 98989880. The comparisons between neighbours: 9 + 89 = 989 > 899; 89 + 898 = 89898 > 89889; 898 + 8 = 8988 > 8898; 8 + 0 = 80 > 08. $\nu(89) = 89/99 \approx 0.898990$ and $\nu(898) = 898/999 \approx 0.898899$, so "89" comes first, as the concatenations say. For [0, 0] the sorted strings start with "0", so the method returns "0", not "00".

Exercise 4.7. State: *reach*, meaning every total 0..*reach* can be paid with the coins used and added so far. If the next coin c satisfies $c \le reach + 1$, use it: *reach* += c (*Theory Lemma 4.7*). Otherwise the total *reach* + 1 cannot be paid, so add the coin *reach* + 1, which extends the range furthest: *reach* = 2 · *reach* + 1. Stop when *reach* ≥ n. Each added coin more than doubles *reach*, so at most about $\log_2 n + 1$ coins are added; the loop is O(m + log n) for m coins. For [1, 5, 10] and n = 20: use 1 (reach 1), add 2 (reach 3), add 4 (reach 7), use 5 (reach 12), use 10 (reach 22). Two coins are added, which a brute force over all choices of added coins confirms.

Exercise 4.8. (1) False as a statement about the algorithm (quicksort is not stable), but it cannot be observed, because equal `int` values are identical. (2) True: object sorts use TimSort, O(n log n) in the worst case. (3) False: the difference can reach $2 \cdot 10^9$, which exceeds $2^{31} - 1$ and overflows. (4) False: boxing and comparator calls made it about five times slower in this book's measurements (410–570 ms against 11–91 ms). (5) False: $\log_2(5!) = \log_2 120 \approx 6.9$, so some input needs at least 7 comparisons. (6) False: some false greedies only fail on larger inputs (Theory review question 3 needed 9 or more intervals); test sizes up to about 10 and prove the rule.

Exercise 4.9. After pass 1: [2, 3, 4, 0, 1, 2, 1]. After pass 2: [2, 2, 1, 0, 1, 2, 1]. Neighbours differ by at most 1 everywhere, and each entry is at most its cap (for example index 2 is 1 ≤ 5, forced down by the 0 at index 3).

Chapter 6 · Binary Search and Searching on the Answer

Exercise 6.1. (1) `a.length`; (2) `lo < hi`; (3) `(lo + hi) >>> 1` (or `lo + (hi - lo) / 2`); (4) `hi = mid`; (5) `lo = mid + 1`. The answer n ("every element is smaller than x") must be inside the range of possible results. With `hi = a.length - 1`, the array [0] and $x = 1$ return 0, although no element is ≥ 1 and the correct answer is 1.

Exercise 6.2. upperBound(a, 8): (lo, hi, mid, a[mid]) = (0, 9, 4, 6) → lo = 5; (5, 9, 7, 11) → hi = 7; (5, 7, 6, 9) → hi = 6; (5, 6, 5, 6) → lo = 6; result 6. lowerBound(a, 6): (0, 9, 4, 6) → hi = 4; (0, 4, 2, 4) → lo = 3; (3, 4, 3, 6) → hi = 3; result 3. The value 6 occurs upperBound(a, 6) − lowerBound(a, 6) = 6 − 3 = 3 times. `Arrays.binarySearch(a, 6)` may return any of 3, 4, 5 (the JDK returns 4 here, its first midpoint); `Arrays.binarySearch(a, 5)` returns −4, because the insertion point of 5 is 3.

Exercise 6.3. The remaining rows are (4, 7, 5, 4 days) → fails, lo = 6; (6, 7, 6, 3 days) → works, hi = 6; then lo = hi = 6 and the loop stops. The answer is 6, with the greedy days [3, 2], [2, 4], [1, 4].

Exercise 6.4. (1) False: it returns *some* matching index; the documentation gives no guarantee which. (2) True: when every element is smaller than x. (3) False: it must be rounded up, or `lo = mid` makes no progress when $hi = lo + 1$. (4) True: each call at least halves the range of candidates (*Theory Theorem 6.4*). (5) False: neighbouring doubles near 10^9 are about $1.2 \cdot 10^{-7}$ apart, so the gap may never drop below 10^{-9}; use a fixed number of iterations. (6) False: *Theory Theorem 6.10* shows it is always an element. (7) False in general: a huge hi can overflow sums or products inside the check (for example `mid * count`), which turns a correct predicate into a wrong one.

Exercise 6.5. ok(T) = "$\Sigma \lfloor T/t[i] \rfloor \geq M$". Each term is non-decreasing in T, so if T works, $T + 1$ works. $lo = 0$ (or 1) and $hi = \min(t) \cdot M$: the fastest machine alone makes M products in that time, so ok(hi) is true. One check is O(n); the search makes about $\log_2(10^{18}) \approx 60$ checks, so O(60 n) ≈ $1.2 \cdot 10^7$ steps. Overflow: hi can be 10^{18}, so the bounds are `long`, and the sum $\Sigma\, T/t[i]$ can exceed `long` when T is large and many $t[i]$ are small; stop adding as soon as the sum reaches M. On t = [3, 2, 5], $M = 10$, with $lo = 0$ and $hi = 20$: (0, 20, 10, 10 products) → hi = 10; (0, 10, 5, 4) → lo = 6; (6, 10, 8, 7) → lo = 9; (9, 10, 9, 8) → lo = 10. The answer is $T = 10$: after 10 minutes the machines have made 3 + 5 + 2 = 10 products, after 9 only 8.

Exercise 6.6. The smallest failing input is a = [1] with $k = 1$ (correct answer 1). Then $lo = 0$, $hi = 1$ and the rounded-down midpoint is 0; `piecesAtLeast(a, 0)` = 1 ≥ k, so `lo = mid` sets lo to 0 again and the loop repeats for ever. In general the method hangs as soon as the range has two values, because lo always passes. Fix: `long mid = lo + (hi - lo + 1) / 2;`.

Exercise 6.7. count(5) = 9, count(6) = 11, count(7) = 11, count(8) = 13, count(10) = 15. The search: (1, 20, 10, 15) → hi = 10; (1, 10, 5, 9) → lo = 6; (6, 10, 8, 13) → hi = 8; (6, 8, 7, 11) → hi = 7; (6, 7, 6, 11) → hi = 6; answer 6. The sorted table is [1, 2, 2, 3, 3, 4, 4, 4, 5, 6, 6, 8, 8, 9, 10, 12, 12, 15, 16, 20], whose 11th element is 6. The search looks for the *smallest* x with count(x) ≥ 11; 7 is not an entry of the table, so count does not change between 6 and 7.

Exercise 6.8. (1) $lo = 1$, $hi = 10^9$ (the largest pile); bounds fit in `int` but the total of hours needs `long`; 30 iterations. (2) lo = 500 at most (the heaviest package), hi = the total ≤ $2.5 \cdot 10^7$; `int` would do but `long` is the safe default; $\lceil \log_2 2.5 \cdot 10^7 \rceil = 25$ iterations. (3) lo = max ≤ 10^6, hi = total ≤ 10^9; at most 30 iterations; sums inside the check reach 10^9, close to the `int` limit, so use `long`. (4) A fixed number of iterations: $10^9 / 10^{-6} = 10^{15} \approx 2^{50}$, so 50 halvings are enough in exact arithmetic; use 100 to be safe.

Exercise 6.9. Compare with *Theory Listing 6.4*. `firstTrue` uses `mid = lo + (hi - lo) / 2` with `hi = mid` / `lo = mid + 1`; `lastTrue` uses `mid = lo + (hi - lo + 1) / 2` with `lo = mid` / `hi = mid - 1`. The rounding must keep *mid* strictly away from the end that its branch assigns, so that every branch shrinks the range.

Chapter 7 · Monotonic Stacks

Exercise 7.1. (1) `Arrays.fill(res, -1)`; (2) `top > 0`; (3) `a[st[top - 1]] < a[i]`; (4) `res[st[--top]] = i`; (5) `st[top++] = i`. For next greater or equal, pop on `a[st[top - 1]] <= a[i]`; for next smaller, pop on `a[st[top - 1]] > a[i]`.

Exercise 7.2. Popped at each step: none, none, none, {2, 1}, {3, 0}, none, {5, 4}. Stacks after: [0], [0, 1], [0, 1, 2], [0, 3], [4], [4, 5], [6]. The result is [4, 3, 3, 4, 6, 6, −1] with 6 pops. Each index is pushed once and can be popped at most once, so 7 pushes allow at most 7 pops; the last index is never popped here.

Exercise 7.3. Rows 2–4: (left 1, right 3, count 1 · 1 = 1, contribution 2); (left 1, right 4, count 2 · 1 = 2, contribution 4); (left −1, right 5, count 5 · 1 = 5, contribution 5). The total is 3 + 6 + 2 + 4 + 5 = 20, and the counts 1 + 6 + 1 + 2 + 5 add to 15 = 5 · 6/2, the number of subarrays. A brute force also gives 20. Index 1 does not own any subarray that also contains index 4, for example [1, 2, 2, 1] or the whole array: the value 1 occurs again at index 4, and the rightmost minimum owns the subarray, so index 4 does.

Exercise 7.4. L = [−1, 0, 0, 2, −1, 4], R = [4, 2, 4, 4, 6, 6], widths [4, 1, 3, 1, 6, 1], areas [8, 4, 9, 5, 6, 3]. The largest rectangle has area 9: height 3 over bars 1..3.

Exercise 7.5. (1) False: each index is pushed once and popped at most once, so the total number of pops is at most n and the method is O(n). (2) True: everything ≥ $a[i]$ is removed, so the top is strictly smaller. (3) True: such a subarray is owned by every position of its minimum. (4) True: the sum of differences is the difference of the sums. (5) False: it changes no answer, only adds work (Theory Chapter 7, review question 4). (6) False: the stack is nondecreasing, so the best deletion is the last character; they come off the end.

Exercise 7.6. The span needs the previous strictly greater price: pop while `p[top] <= p[i]`; then the span is $i + 1$ if the stack is empty and i − top otherwise; push i. O(n) time and memory, by the push–pop count. For [30, 25, 27, 27, 40, 22, 26] the spans are [1, 1, 2, 3, 5, 1, 2].

Exercise 7.7. Any array with a repeated minimum fails; the smallest positive one is [1, 1]. The true sum is 1 + 1 + 1 = 3; the buggy method gives 4, because the subarray [1, 1] is owned by both positions. On [3, 1, 2, 2, 1] it gives 24 instead of 20. Fix: make one side non-strict, for example `while (r < n && a[r] > a[i]) r++;` on the right.

Exercise 7.8. 3: push, stack 3, k 3. 1: pop 3, stack 1, k 2. 4: push, stack 14. 1: pop 4, stack 11, k 1. 5: push, stack 115. 9: push, stack 1159. 2: pop 9, stack 1152, k 0. The result is 1152. Spending the third deletion on the 9 gives 1152; spending it on the 5 would give 1192. The two first differ at the third digit, where 5 < 9, so 1152 is smaller. The greedy deletes the 9 because 9 > 2 is the first descent of 11592.

Exercise 7.9. Compare with *Theory Listing 7.5*. The left scan pops on `a[top] >= a[i]` (so left is strictly smaller) and the right-to-left scan pops on `a[top] > a[i]` (so right is smaller or equal); the opposite pairing (non-strict on the left, strict on the right) is also correct. The same strictness on both sides is never correct.

Chapter 10 · Number Theory for Programmers

Exercise 10.1. (1) `1 % m`; (2) `(exp & 1) == 1`; (3) `base = base * base % m`; (4) `exp >>= 1`; (5) `inverse(fact[n])`, that is `power(fact[n], MOD - 2, MOD)`; (6) `invFact[i] * i % MOD`. Line (6) uses $(i-1)!^{-1} = i \cdot (i!)^{-1}$, because $(i-1)! \cdot i \cdot (i!)^{-1} = i! \cdot (i!)^{-1} \equiv 1$ (*Theory Theorem 10.10*). It needs $n <$ MOD so that $n!$ has an inverse.

Exercise 10.2.

iteration	a	b	q	r
1	2024	1717	1	307
2	1717	307	5	182
3	307	182	1	125
4	182	125	1	57
5	125	57	2	11
6	57	11	5	2
7	11	2	5	1
8	2	1	2	0

After the 8th iteration $b = 0$ and $a = 1$, so gcd(2024, 1717) = 1: the numbers are coprime ($2024 = 2^3 \cdot 11 \cdot 23$ and $1717 = 17 \cdot 101$). Eight iterations. lcm = 2024 / 1 · 1717 = 3 475 208.

Exercise 10.3. For the first blank row, the row above gives $x' = -5$ and $y' = 26$, so $x = y' = 26$ and $y = x' - q \cdot y' = -5 - 2 \cdot 26 = -57$. The remaining rows:

call	q	x	y	check
(125, 57)	2	26	−57	125 · 26 − 57 · 57 = 1
(182, 125)	1	−57	83	−10 374 + 10 375 = 1
(307, 182)	1	83	−140	25 481 − 25 480 = 1
(1717, 307)	5	−140	783	−240 380 + 240 381 = 1
(2024, 1717)	1	783	−923	1 584 792 − 1 584 791 = 1

So $1717 \cdot (-923) \equiv 1 \pmod{2024}$, and the inverse is −923 + 2024 = 1101. `modInverse(1717, 2024)` returns 1101.

Exercise 10.4. $45 = 101101_2$, so the loop runs 6 times.

exp at start	bit	r after	base after	exp after
45	1	7	49	22
22	0	7	401	11
11	1	807	801	5
5	1	407	601	2
2	0	407	201	1
1	1	807	401	0

$7^{45} \bmod 1000 = 807$ (Java's `BigInteger.modPow` agrees). The naive loop uses 44 multiplications; fast power uses 10 (6 squarings and 4 multiplications into r).

Exercise 10.5. Every even x has spf 2. The odd composites are 9 → 3, 15 → 3, 21 → 3, 25 → 5, 27 → 3, 33 → 3, 35 → 5 and 39 → 3. The primes (spf[x] = x) are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31 and 37.

756 (spf 2) → 378 (spf 2) → 189 (spf 3) → 63 (spf 3) → 21 (spf 3) → 7 (spf 7) → 1, so $756 = 2^2 \cdot 3^3 \cdot 7$, with (2 + 1)(3 + 1)(1 + 1) = 24 divisors and kernel 3 · 7 = 21. $84 = 2^2 \cdot 3 \cdot 7$, with 12 divisors and kernel 21. The kernels are equal, so 756 · 84 is a square: 63 504 = 252^2. The kernel of $1176 = 2^3 \cdot 3 \cdot 7^2$ is 2 · 3 = 6 ≠ 21, so 756 · 1176 is not a square.

Exercise 10.6. (1) False: 6 / 2 = 3 ≡ 3 (mod 4), but (6 mod 4) / (2 mod 4) = 2 / 2 = 1. Divide by multiplying with an inverse. (2) False: for $a = 3$, $3^{9998} \bmod 10\,000 = 8889$ and 3 · 8889 ≡ 6667; Fermat needs a prime modulus. (3) True: with $b = 0$ the loop does not run and returns $|a|$; gcd(0, 12) runs one iteration and returns 12. (4) True: at most 2 000 000 012 < 2 147 483 647. (5) False: 12 and 18 both have prime set {2, 3}, but 12 · 18 = 216 is not a square; compare kernels (3 and 2). (6) False: the inner loop runs only for primes, and $\Sigma\, 1/p$ grows like ln ln n, so the cost is O(n log log n). (7) True: $10^7 \cdot 4$ bytes = 40 MB.

Exercise 10.7. Key idea: $a[i] \cdot a[j]$ is a square exactly when ker($a[i]$) = ker($a[j]$), so count values by kernel. Build an SPF sieve to 10^7 (40 MB, O(V log log V)), compute each kernel in O(log V), count kernels in a `HashMap<Integer, Integer>` (or a second `int[]` if memory allows), and answer $\Sigma\, c(c-1)/2$ over the counts c, in `long`. Total O(V log log V + n log V). For the example the kernels are 2, 2, 3, 3, 2, 5, 2, 3, 1, 1: kernel 2 occurs 4 times (6 pairs), kernel 3 three times (3 pairs), kernel 1 twice (1 pair), kernel 5 once. The answer is 10, which matches a brute force over all 45 pairs.

Exercise 10.8. `r * base` and `base * base` are computed in `int` and overflow as soon as a product exceeds 2 147 483 647. With exponent 2 the first failing base is 46 341 ($46\,341^2 = 2\,147\,488\,281$). For example `powerBuggy(100000, 2, 1_000_000_007)` returns 410 065 401, while the correct value $10^{10} \bmod (10^9 + 7)$ is

999 999 937. Fix: declare `r` and `base` as `long` (as in *Theory Listing 10.3*) and cast the result, or write `r = (int) ((long) r * base % m)` and `base = (int) ((long) base * base % m)`.

Exercise 10.9. (1) Factorials and inverse factorials up to 10^6 (*Theory Listing 10.5*): O(10^6) once, O(1) per query, two `long` arrays of 8 MB each. Trap: reduce after each of the two multiplications in `nCr`. (2) Pascal's triangle (*Theory Listing 10.6*): about $5 \cdot 10^5$ additions and 1001^2 entries, 8 MB as `long` (4 MB as `int`). Trap: Fermat inverses do not exist modulo 10 000. (3) Boolean sieve to 10^7: about $2.3 \cdot 10^7$ steps and 10 MB, then O(1) per number. Trap: 0 and 1 are not prime. (4) Trial division up to $\sqrt{(10^{12})} = 10^6$ iterations. Trap: the cofactor left at the end is a prime factor too. (5) Fast power: 60 iterations. Trap: reduce the base first and keep products in `long`. (6) Divide out the 168 primes below 1000, then test the cofactor for being a square (*Theory Listing 10.12*): at most 168 divisions per value, under $2 \cdot 10^7$ in total. Trap: use an exact integer square root.

Chapter 13 · Dynamic Programming III: Increasing Subsequences and Extra Dimensions

Exercise 13.1. (1) `len`; (2) `>=`; (3) `tails[lo] = x`; (4) `if (lo == len) len++`. For the non-decreasing version blank (2) becomes `>`: equal values may follow each other, so x must go after every tail equal to it, which is the first tail strictly greater than x (the upper bound).

Exercise 13.2.

x	position p	p + 1	tails after
4	0	1	[4]
10	1	2	[4, 10]
4	0	1	[4, 10]
3	0	1	[3, 10]
8	1	2	[3, 8]
9	2	3	[3, 8, 9]
1	0	1	[1, 8, 9]
5	1	2	[1, 5, 9]

The LIS length is 3; *Theory Listing 13.3* returns 3, 8, 9 (4, 8, 9 also works). The final `tails` = [1, 5, 9] is not a subsequence: 1 and 5 come after 9 in a. The second 4 lands on the existing 4 and changes nothing, which is the strict version at work. The non-decreasing version on the same array gives length 4 (for example 4, 4, 8, 9).

Exercise 13.3. up = [1, 2, 2, 3, 3, 1, 4, 2] and down = [2, 3, 2, 3, 2, 1, 2, 1]. A peak is allowed at indices 1, 2, 3, 4 and 6, with up + down − 1 = 4, 3, 5, 4, 5. The longest mountain has length 5, so the fewest removals are 8 − 5 = 3. Two examples: 2, 5, 6, 4, 1 (peak 6 at index 3) and 2, 3, 6, 7, 2 (peak 7 at index 6). Indices 0, 5 and 7 cannot be peaks because one side has length 1.

Exercise 13.4. Ranks [3, 1, 5, 1, 2, 4], $V = 5$.

a[i]	rank	prefixMax(rank − 1)	cur	fw[1..V] after
30	3	0	1	[0, 0, 1, 1, 0]
10	1	0	1	[1, 1, 1, 1, 0]
50	5	1	2	[1, 1, 1, 1, 2]
10	1	0	1	[1, 1, 1, 1, 2]
20	2	1	2	[1, 2, 1, 2, 2]
40	4	2	3	[1, 2, 1, 3, 2]

The answer is 3 (10, 20, 40). The second 10 has rank 1, and its query covers ranks 1..0, an empty range, so it cannot extend the first 10: in a strictly increasing subsequence equal values may not follow each other. Note how raising rank 3 also wrote node 4, and raising rank 1 wrote nodes 1, 2 and 4.

Exercise 13.5. (1) False: for [3, 4, 1] it ends as [1, 4], which is not a subsequence. (2) True: equal values may follow each other, so x goes after all tails equal to it. (3) False: the peak may not be the first element of the mountain, so up[i] ≥ 2 is required. (4) False: a node stores a maximum and cannot recover the second-largest value after the largest is lowered. (5) True: then two envelopes of equal width can never both be in a strictly increasing run of heights. (6) True: every state of the plain DP is split into K states, and each transition is done once per remainder.

Exercise 13.6. Sort by w ascending and, for equal w, by h descending, so that two blocks of equal width can never both be chosen. Compress the heights to ranks. The Fenwick tree is indexed by height rank and stores, at each node, the largest tower value ending with a height in its range (a `long` sum, not a length). For each block, query the prefix maximum over ranks below its own (strict), add v, and raise its rank. Time O(n log n), memory O(n). For the six blocks the answer is 20, for example (1, 3, 4) + (2, 4, 3) + (3, 5, 6) + (4, 6, 7) = 4 + 3 + 6 + 7 = 20; a brute force over all subsets agrees.

Exercise 13.7. The query starts at `x` instead of `x - 1`, so it includes elements with the **same** rank and builds a non-decreasing subsequence. For ranks [1, 1] (two equal values) it returns 2 instead of 1. Fix: `for (int i = x - 1; i > 0; i -= i & -i)`.

Exercise 13.8. (1) The quadratic DP of *Theory Listing 13.1* with "$a[i]$ % $a[j]$ == 0" in place of "<": about $3000^2/2 = 4.5 \cdot 10^6$ pairs. (2) Patience sorting, *Theory Listing 13.2*: about $2 \cdot 10^5 \cdot 18 \approx 3.5 \cdot 10^6$ comparisons. (3) Compression plus a Fenwick tree of maximum sums (`long`), *Theory Listing 13.7*: O(n log n). (4) State (last value, XOR so far); since values are below 1024, there are 1024 XOR states, each with a Fenwick tree over values: about $1000 \cdot 1024 \cdot 10 \approx 10^7$ steps for the queries and as many for the updates, about $2 \cdot 10^7$ in all. (5) Brute force over all $2^{20} \approx 10^6$ subsets, checking each in O(n): about $2 \cdot 10^7$ steps.

Exercise 13.9. Compare with *Theory Listing 13.4*. The two places that matter are the comparison `strict ? tails[mid] >= x : tails[mid] > x` and the line `end[i] = lo + 1` after the update.

APPENDIX C

Error log

Every mistake goes here with the rule that would have prevented it. Causes: misread statement, wrong idea, overflow / mod, off-by-one, TLE / complexity, input reading, edge case, stack overflow, ran out of time. Re-solve each entry three days later.

Date	Problem	Topic	Cause	Rule that prevents it	Re-solve

Date	Problem	Topic	Cause	Rule that prevents it	Re-solve

Date	Problem	Topic	Cause	Rule that prevents it	Re-solve

APPENDIX D

Mock scores and SP checkpoints

Copy each mock's result here so the trend is visible on one page. The checkpoints are estimates of what is on track for the Specialist Programmer role, not Infosys's published cut-offs (it publishes none).

Mock	Day	On track if	Q1 %	Q2 %	Q3 %	Q4 %	Score	Weakest topic
Mock 1 · Phase 1 topics	9	Q1 full; Q2 ≥ half						
Mock 2 · number theory and DP added	16	Q1+Q2 full; ≥ 50						
Mock 3 · four problems	23	≥ 60; brute force on all						
Mock 4 · official Infosys sample paper	25	grade, then walk-throughs						
Mock 5 · dress rehearsal	27	≥ 65						

If you are below the checkpoint after…	Do this
Mock 1 (Day 9)	Skip bonus problems in week 2 and redo the core problems of your weakest day.
Mock 2 (Day 16)	Same, and spend the review block on DP problems from Days 11–14.
Mock 3 (Day 23)	Days 24–26: repeats and error-log repair only, no new topics.
Mock 5 (Day 27)	On exam day, lock in Q1 and Q2 first and use partial marks on Q3.

APPENDIX E

Notes and scratch paper